

Bachelor of Science in Computer Science
UNIGE - Faculty of Science - Department of Computer Science

Structural Sensitivity of Convolutional Networks to Fourier Basis Directions

Author:
David Tournier Marrucho

Supervisor:
Stephane Marchand Maillet

October 11, 2026

Contents

Table of Notation **5**

1 Introduction **7**

2 Background **9**
2.1 Convolutional Networks in Computer Vision 9
2.2 Robustness of Vision Models 9
2.3 Adversarial Perturbations 10
2.4 Universal Perturbations and Directions of Vulnerability 10

3 Theoretical Foundations: Convolution and the Fourier Basis **11**
3.1 Objective of This Section 11
3.2 Fourier and Vectorization Conventions 11
3.2.1 1D Fourier Basis and Discrete Transform 11
3.2.2 2D Fourier Basis 12
3.2.3 Vectorization and the Kronecker Product 14
3.3 Discrete Convolution in Dimension 1 15
3.4 Fourier Functions as Eigenfunctions of Circular Convolution 16
3.4.1 Consequence: Diagonalization of Circulant Matrices 17
3.5 Circular Convolution in Dimension 2 18
3.6 Matrix Representation of a 2D Convolution 19
3.6.1 Diagonalization by the Kronecker Product of Fourier Bases 20
3.7 Extension to the Multichannel Case 20
3.7.1 Channel Mixing at a Fixed Frequency 20
3.7.2 Global Matrix Form 21
3.8 Linear Stacking of Convolutions 23
3.8.1 Simplification by Telescoping Transformations 24
3.8.2 Frequency Decoupling and the Mixing Matrix $G^{(w)}$ 24
3.8.3 Sensitivity Analysis Using Singular Value Decomposition (SVD) 25
3.8.4 Proof: Structure of the Singular Vector Characterizing Sensitivity 25
3.8.5 Verification of Orthonormality 27
3.8.6 Final Expression for the Fourier Pattern 27
3.9 Skip Connections and Normalization Layers 28
3.9.1 What Is a *Skip Connection*? 28
3.9.2 The Case of a Skip Connection 29
3.9.3 The Case of Normalization Layers 30
3.9.4 Conclusion 31
3.10 Downsampling Layers: Stride and Average Pooling 31
3.10.1 Spatial Subsampling and Frequency Aliasing 32
3.10.2 Strided Convolution and Spectral Folding 34
3.10.3 Composition with a Convolution of Stride 1 36
3.10.4 Construction of Right Singular Vectors 37
3.10.5 Average Pooling as a Special Case 39

3.11 Real-Valued Image Constraint . . . 40
3.11.1 Hermitian Symmetry of Fourier Coefficients . . . 40
3.11.2 Consequence for Fourier Perturbations . . . 41
3.12 Limitations and Scope of the Theoretical Framework . . . 42
3.12.1 An Exact Result in a Controlled Setting . . . 42
3.12.2 Differences from Practical Architectures . . . 43
3.12.3 Local Scope of Frequency Analysis in Nonlinear CNNs . . . 43
3.12.4 Practical Scope of the Result . . . 45

4 Experimental Analysis of CNN Frequency Sensitivity 46
4.1 Chapter Objective and Scope . . . 46
4.2 Experimental Setup . . . 46
4.2.1 Datasets . . . 46
4.2.2 Architecture . . . 47
4.2.3 Training Parameters . . . 48
4.2.4 Baseline Performance . . . 48
4.3 Constructing Fourier Perturbations . . . 48
4.3.1 Single Fourier Attack . . . 49
4.3.2 Amplitude Control . . . 50
4.4 Measuring Frequency Sensitivity . . . 50
4.4.1 Definition of the Fool Ratio . . . 51
4.4.2 Generating Heatmaps . . . 51
4.4.3 Selecting the Most Sensitive Frequency . . . 51
4.5 Experimental Results . . . 52
4.5.1 Frequency Sensitivity Heatmaps . . . 52
4.5.2 Most Sensitive Frequencies . . . 53
4.5.3 Stability of Sensitive Regions Across Training Runs . . . 53
4.5.4 Internal Amplification of Sensitive Frequencies . . . 55
4.5.5 Comparison with Random Noise . . . 56
4.5.6 Effect of Perturbation Amplitude . . . 57
4.5.7 Connection with the Theoretical Framework . . . 58
4.6 Chapter Summary . . . 59

5 Conclusion 60

A Matrix Representation of a Linear Map 62

B Eigenvectors, Eigenvalues, and Diagonalization 64

C Supplementary Material 66
C.1 Periodicity, Boundaries, and the Gibbs Phenomenon . . . 66
C.2 Detailed Proof of the BCCB Structure . . . 67
C.3 Detailed Proof for Skip Connections . . . 69
C.4 Graphical Representation of Stride . . . 71
C.5 Illustration of Average Pooling . . . 71
C.6 Review of the SVD and Amplification Directions . . . 72
C.7 Mean Fourier Spectrum of Clean Images . . . 73
C.8 Supplementary Analysis: The Role of Channel Mixing in SFA . . . 74

C.9 Internal Heatmaps by Layer 78
C.10 Further Analysis of Logit Margins and Local Boundaries 78
C.10.1 Distribution of Logit Margins 78

Table of Notation

Notation	Description
Indices, Images, and Frequencies	
N	Spatial size of the images considered. In the CIFAR experiments, $N = 32$.
$\mathbb{Z}_N$	Set of indices $\{0, \dots, N-1\}$, interpreted modulo N.
(m, n)	Spatial coordinates of a pixel: m for the vertical direction and n for the horizontal direction.
(u, v)	2D Fourier frequency: u for the vertical frequency and v for the horizontal frequency.
$w(u, v)$	Flattened index associated with frequency (u, v), used after vectorization.
x, X, Y	Input image or signal, matrix-form input image, and output image.
Fourier Analysis and Convolution	
h	Convolution filter.
$h_{i,j}$	Filter connecting input channel j to output channel i.
$*$	Circular convolution.
ϕ_k	1D discrete Fourier vector associated with frequency k.
$\Phi_{u,v}$	2D Fourier mode associated with frequency (u, v).
F_N	1D Fourier matrix whose columns are the vectors ϕ_k.
Q_N	Vectorized 2D Fourier matrix, defined by $Q_N = F_N \otimes F_N$.
$\mathrm{vec}(X)$	Vectorization of image X by stacking its columns.
λ_f, $\lambda_{u,v}^{i,j}$	Eigenvalues associated with the action of a convolution filter on a Fourier frequency.
Matrix Representation of the Network	
I_r	Identity matrix of size $r \times r$. In particular, I_N denotes the identity of size N, and I_{m_k} the identity acting on the channels of layer k.
M	Matrix representing a convolutional layer, or the global linear operator, depending on the context.
$M^{(k)}$	Matrix representing the k-th convolutional layer from $\mathbb{C}^{m_k N^2}$ to $\mathbb{C}^{m_{k+1} N^2}$.
m_{in}, m_{out}	Number of input and output channels of a convolutional layer considered in isolation.
m_k	Number of input channels of layer k.
L	Matrix describing channel mixing in the Fourier domain.

Notation	**Description**
$G^{(w)}$	Mixing matrix associated with a given frequency w.
σ	Singular value, used to measure the amplification of an input direction.
a, b	Right and left singular vectors associated with a given frequency.
Perturbations and Experimental Protocol	
ϵ	Maximum permitted perturbation amplitude, measured in the ℓ_∞ norm.
$p_{u,v}$	Real-valued spatial pattern constructed from frequency (u, v).
$\delta_{u,v}$	Normalized Fourier perturbation associated with frequency (u, v).
$\tilde{x}$	Perturbed image obtained by adding $\delta_{u,v}$ and then clipping to $[0, 1]$.
f	Classification model under study.
$\mathcal{D}$	Set of images used for evaluation.
$\mathrm{FR}(u, v)$	*Fool ratio* obtained at frequency (u, v), i.e., the proportion of predictions that change.
H	Frequency sensitivity heatmap containing the values of $\mathrm{FR}(u, v)$.
$(u^\star, v^\star)$	Frequency that maximizes the *fool ratio*.

1 Introduction

Convolutional neural networks, often abbreviated as CNNs, are inspired by the vertebrate visual cortex and play a central role in computer vision. They are used to recognize objects, classify images, detect shapes, and automatically extract visual information.

Their effectiveness stems largely from their ability to learn local patterns in images and progressively combine them into more abstract representations.

Vulnerabilities and Adversarial Attacks

Despite their strong performance, these models can exhibit structural vulnerabilities. An image can be modified slightly, in a way that is imperceptible to the human eye, yet causes a substantial change in the network's prediction. This phenomenon is known as an **adversarial perturbation** [13].

A particularly interesting aspect of these perturbations is that they can sometimes be universal. In some cases, a perturbation can be found that is based directly on the network's architecture (how it operates), exploiting a structural weakness in its internal workings. Once found, such a perturbation can be applied to any image, **regardless** of its content, and cause the model to make incorrect predictions on a substantial proportion of images. This is called a **universal** adversarial perturbation [10].

The central question is then where this sensitivity comes from. One possible explanation attributes it to the training data or model optimization. In this work, we explore another possibility: **the very structure of convolutional networks**. CNNs rely on convolutional layers, which have specific mathematical properties.

The Fourier Basis Approach

To investigate this question, we use the Fourier basis. An image can be described pixel by pixel, but it can also be decomposed into frequency patterns: slow variations, rapid oscillations, and horizontal, vertical, or diagonal patterns. These frequency patterns provide another way to describe the structures present in an image.

Analyzing an image not only in the spatial domain, through its pixels, but also in the frequency domain therefore helps us understand which image components influence the network's behavior and which perturbation directions may become particularly sensitive.

This connection is the starting point of the work by Tsuzuku and Sato [14], who show that certain convolutional networks are particularly sensitive to directions drawn from the Fourier basis. Their idea is that some universal adversarial perturbations can be better understood by studying how convolutional layers process frequencies.

How does the convolutional architecture of CNNs give rise to preferred directions of vulnerability in the Fourier domain, and to what extent does this structural sensitivity help us understand the existence of universal adversarial perturbations?

The goal of this report is to revisit this idea and reconstruct it step by step in greater detail, introducing the concepts needed to understand the results so that any reader with a basic knowledge of linear algebra and machine learning can understand the implications of this work.

Report Organization

To address this question, the report is organized as follows:

- **Chapter 2:** General background on convolutional networks, robustness, and adversarial perturbations.
- **Chapter 3:** Theoretical foundations connecting convolution and the Fourier basis. In particular, this chapter shows how Fourier functions emerge naturally in linear analysis.
- **Chapter 4:** Experimental reproduction, the protocol used, and frequency sensitivity *heatmaps*, followed by a discussion of the results and limitations.
- **Conclusion:** Summary of the findings and directions for future research.

2 Background

This chapter establishes the concepts needed for the rest of the report: CNNs as convolution-based models, robustness as prediction stability, and adversarial perturbations as directions in image space.

2.1 Convolutional Networks in Computer Vision

A digital image can be viewed as a tensor $X \in \mathbb{R}^{C\times H\times W}$, where C denotes the color channels and H, W the spatial dimensions. A classifier learns a function

$$f : \mathbb{R}^{C\times H\times W} \longrightarrow \{1, \ldots, K\},$$

that assigns each image to one of K categories.

Convolutional networks exploit the spatial structure of images. Unlike a fully connected layer, a convolution applies the same local filter at every position. This weight sharing enables a pattern, such as an edge or a texture, to be detected regardless of its position.

By stacking several layers, the network combines local patterns into more abstract representations. The first layers capture simple structures, while deeper layers produce information more closely related to objects. Figure 2.1 illustrates this principle.

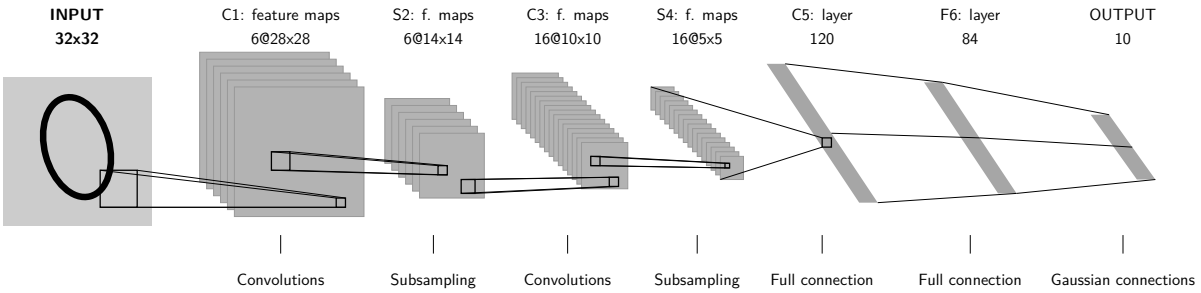


Figure 2.1: Architecture of the LeNet-5 convolutional neural network. Adapted from [8].

The output of size 10 here corresponds to a prediction among the ten possible digits.

2.2 Robustness of Vision Models

Performance is often measured on clean images similar to those used during training. This measure is insufficient: a model can be accurate yet unstable under small variations.

Consider an image x and a perturbation δ. A model is locally robust if its prediction remains stable when x is replaced by $x + \delta$:

$$f(x + \delta) = f(x) \qquad \text{for perturbations } \delta \text{ that are sufficiently small.}$$

2.3 Adversarial Perturbations

An adversarial perturbation is a modification chosen to change a model's prediction while remaining small or barely visible to the human eye [13]. It is not random noise: it follows a direction that strongly influences the network's output.

Starting from an image x, we construct a perturbed image

$$\widetilde{x} = x + \delta,$$

with a perturbation δ that is sufficiently small but satisfies

$$f(\widetilde{x}) \neq f(x).$$

In this report, these perturbations primarily serve as an analytical tool: they indicate which directions in image space are strongly amplified by the model.

2.4 Universal Perturbations and Directions of Vulnerability

In the simplest case, an adversarial perturbation is constructed for a given image and therefore depends strongly on that image.

A universal adversarial perturbation is based on a stronger idea: finding a single perturbation δ that can be added to many different images [10].

Such a perturbation therefore exploits more than a particular feature of a specific image. It exploits a sensitive direction of the model, related to the learned weights, the data, and nonlinearities, but also to the structure of the architecture. This last point motivates the present report.

If an image is viewed as a point in a high-dimensional space, a perturbation is a displacement. Some directions have little effect, whereas others quickly cross a decision boundary, as illustrated in Figure 2.2.

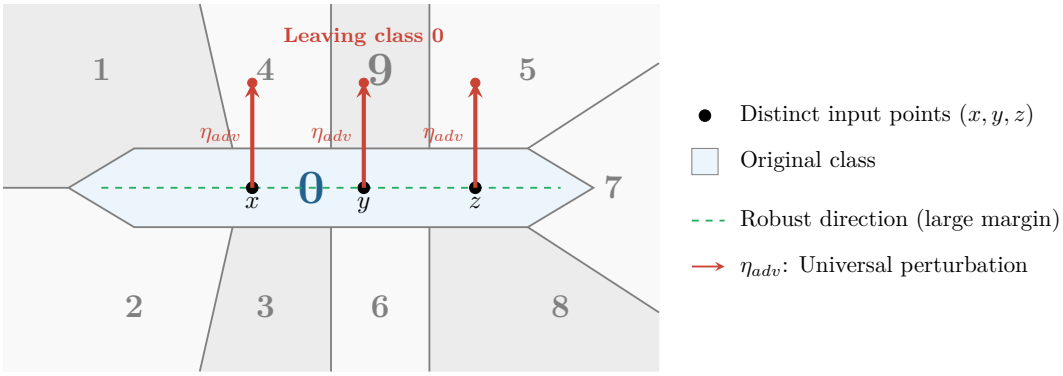


Figure 2.2: Illustration of a universal perturbation in an anisotropic decision region. The same perturbation direction η_{adv} is applied to several distinct input points x, y, and z, moving them outside the region associated with class 0.

3 Theoretical Foundations: Convolution and the Fourier Basis

3.1 Objective of This Section

This section aims to reconstruct the theoretical reasoning proposed by Tsuzuku and Sato [14]. The aim is to understand why convolutional networks are particularly sensitive to certain directions of the Fourier basis by mathematically characterizing these vulnerabilities.

The general idea rests on the mathematical structure of convolutional layers. To study this structure, we adopt a simplified linear framework, setting aside the components commonly introduced into neural networks to break linearity (such as nonlinear activation functions and *max-pooling*).

The following developments use two standard linear algebra results: the matrix representation of a linear map and the relationship between eigenvectors, eigenvalues, and diagonalization. These results are reviewed in Appendices A and B.

3.2 Fourier and Vectorization Conventions

Before studying convolutions, we establish the conventions used throughout this chapter.

In what follows, all frequencies and spatial indices are interpreted modulo N.

3.2.1 1D Fourier Basis and Discrete Transform

For an integer $N \geq 1$, let

$$\mathbb{Z}_N = \{0, \dots, N-1\}$$

denote the set of indices taken modulo N. In concrete terms, this means that index N is identified with index 0, index $N+1$ with index 1, and so on. This convention is used throughout the chapter.

For each frequency $k \in \mathbb{Z}_N$, we define the normalized discrete Fourier vector

$$\phi_k[n] = \frac{1}{\sqrt{N}} e^{2\pi i k n/N}, \qquad n \in \mathbb{Z}_N. \tag{3.1}$$

The vector ϕ_k represents a discrete wave of frequency k. Small values of k correspond to slow variations, while higher frequencies correspond to faster oscillations over the N signal points. The factor $1/\sqrt{N}$ normalizes the vector, giving

$$\|\phi_k\|_2 = 1.$$

We collect all these vectors in a single matrix:

$$F_N = \begin{pmatrix} | & | & & | \\ \phi_0 & \phi_1 & \cdots & \phi_{N-1} \\ | & | & & | \end{pmatrix}. \tag{3.2}$$

The matrix F_N therefore contains, column by column, all possible Fourier directions for a signal of size N. With the chosen normalization, its columns form an orthonormal basis of $\mathbb{C}^N$, that is,

$$F_N^H F_N = I_N, \qquad F_N^{-1} = F_N^H.$$

The discrete Fourier transform then consists of expressing a signal $x \in \mathbb{C}^N$ in this basis. Under our convention, the Fourier coefficients of x are given by

$$\widehat{x} = F_N^H x, \tag{3.3}$$

or, component by component,

$$\widehat{x}[k] = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} x[n] e^{-2\pi i k n/N}. \tag{3.4}$$

The coefficient $\widehat{x}[k]$ measures the contribution of frequency k to the signal.

Since F_N is unitary, the signal can be reconstructed from its Fourier coefficients as

$$x = F_N \widehat{x}, \tag{3.5}$$

or, component by component,

$$x[n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \widehat{x}[k] e^{2\pi i k n/N}. \tag{3.6}$$

3.2.2 2D Fourier Basis

We now move from 1D signals to 2D images. For an image $X \in \mathbb{C}^{N \times N}$, we use spatial indices

$$(m, n) \in \mathbb{Z}_N^2.$$

The index m denotes the vertical position, and n the horizontal position.

Similarly, a 2D frequency is described by a pair

$$(u, v) \in \mathbb{Z}_N^2.$$

The index u corresponds to the vertical frequency, while v corresponds to the horizontal frequency.

For each frequency pair (u, v), we define the 2D Fourier image

$$\Phi_{u,v}[m, n] = \phi_u[m]\phi_v[n] = \frac{1}{N} e^{2\pi i(um+vn)/N}. \tag{3.7}$$

This formula means that a 2D Fourier mode is obtained as the product of a vertical wave and a horizontal wave, in matrix form: $\Phi_{u,v} = \phi_u \phi_v^T$.

Varying the frequencies u and v produces different periodic patterns in image space. Visually, these patterns can take the form of stripes, grids, or oriented oscillations whose density depends directly on the pair (u, v). Adding a Fourier direction to an image therefore amounts to superimposing a periodic geometric pattern.

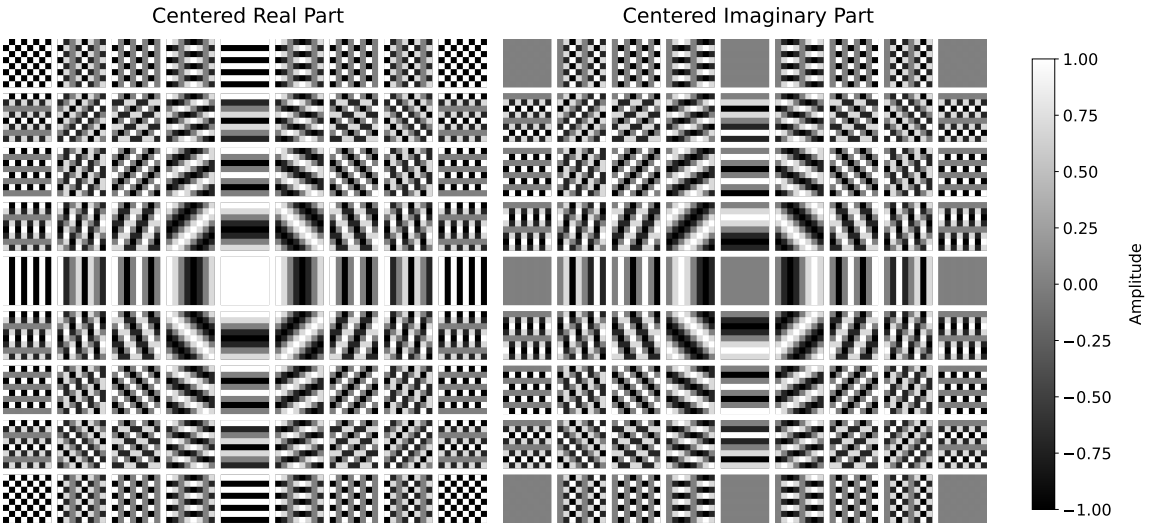


Figure 3.1: Visualization of 2D Fourier basis functions for all frequency pairs (u, v) of an image of size 9×9. The real and imaginary parts are displayed separately. Low frequencies lie at the center of the grids, while high frequencies lie at the edges.

In figures, frequencies are sometimes shown in a centered arrangement, with low frequencies at the center of the image. This convention is convenient for visualization. In the formulas, however, the indices always remain in $\{0, \ldots, N-1\}$ and are interpreted modulo N.

3.2.3 Vectorization and the Kronecker Product

Throughout this chapter, we vectorize images by stacking their columns. For $X \in \mathbb{C}^{N\times N}$, we therefore define

$$\mathrm{vec}(X) = \begin{pmatrix} X[0,0] \\ X[1,0] \\ \vdots \\ X[N-1,0] \\ X[0,1] \\ \vdots \\ X[N-1,N-1] \end{pmatrix}. \tag{3.8}$$

The first column is read first, then the second, and so on. Under this convention, entry $X[m,n]$ corresponds to the vectorized index

$$m + Nn.$$

We also use the Kronecker product, denoted by $\otimes$. It is defined as follows: for $A \in \mathbb{C}^{m\times n}$ and $B \in \mathbb{C}^{p\times q}$, the block matrix of size $mp \times nq$ is defined by

$$A \otimes B = \begin{pmatrix} a_{11}B & a_{12}B & \cdots & a_{1n}B \\ a_{21}B & a_{22}B & \cdots & a_{2n}B \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}B & a_{m2}B & \cdots & a_{mn}B \end{pmatrix}.$$

In particular, it allows us to construct a 2D Fourier basis from two 1D Fourier bases. The algebraic properties of vectorization and the Kronecker product are established results in matrix analysis [5].

Under our column-wise vectorization convention, in particular,

$$\mathrm{vec}(ab^T) = b \otimes a. \tag{3.9}$$

Applying this identity to the Fourier image

$$\Phi_{u,v} = \phi_u \phi_v^T,$$

gives

$$\mathrm{vec}(\Phi_{u,v}) = \phi_v \otimes \phi_u. \tag{3.10}$$

Thus, throughout the remainder of the chapter, the column of Q_N associated with the 2D frequency (u,v) is denoted by

$$q_{u,v} = \phi_v \otimes \phi_u.$$

We then denote by

$$Q_N = F_N \otimes F_N \tag{3.11}$$

the vectorized 2D Fourier basis. Since F_N is unitary,

$$Q_N^H Q_N = I_{N^2}.$$

The columns of Q_N form an orthonormal basis of $\mathbb{C}^{N^2}$.

With the preceding notation, the column associated with frequency (u, v) is

$$(Q_N)_{w(u,v)} = q_{u,v} = \phi_v \otimes \phi_u.$$

The index $w(u, v)$ is simply the flattened index that maps the frequency pair (u, v) to a single column index in Q_N. It is defined by

$$w(u, v) = u + Nv. \tag{3.12}$$

Conversely, given a flattened index $w \in \{0, \dots, N^2 - 1\}$, the corresponding frequency pair is recovered as

$$u = w \bmod N, \qquad v = \left\lfloor \frac{w}{N} \right\rfloor.$$

3.3 Discrete Convolution in Dimension 1

Before studying images, we begin with the simpler case of a periodic discrete signal represented by one period $x = (x[0], x[1], \dots, x[N-1])$ with $x[i] \in \mathbb{R}$. Let $x \in \mathbb{R}^N$ be our signal and $h = (h[0], h[1], \dots, h[N-1]) \in \mathbb{R}^N$ our filter (assumed to have the same size as the signal, with zero-padding if necessary). The circular convolution of h and x is defined as

$$y[n] = (h * x)[n] = \sum_{k=0}^{N-1} h[k]\, x[(n-k) \bmod N] \text{ [15]}$$

The modulo operation means that the signal indices are taken in $\mathbb{Z}_N$. Thus, when an index falls outside the interval $\{0, \dots, N-1\}$, it is mapped back to its corresponding class in $\mathbb{Z}_N$. The signal is therefore interpreted as periodic, as illustrated in Figure 3.2.

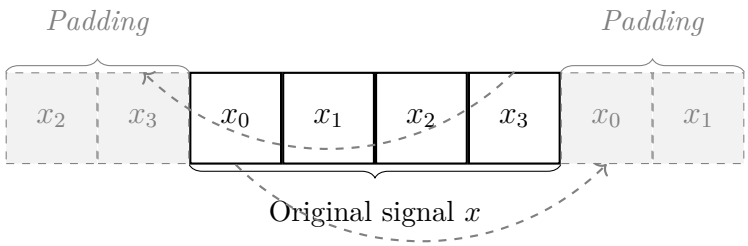


Figure 3.2: Circular *padding* for a 1D signal. The signal boundaries repeat periodically.

Remark on the periodicity assumption. In this section, we work with circular convolutions. This amounts to treating signals as periodically extended: when we pass one end of the signal, we continue from the other end. This assumption is mathematically useful because it allows convolutions to be diagonalized by the discrete Fourier basis.

However, it does not always match the exact boundary behavior of real images: if the ends do not join naturally, periodization can create an artificial discontinuity. This point is discussed in greater detail in Appendix C.1.

3.4 Fourier Functions as Eigenfunctions of Circular Convolution

One-dimensional case. We work in the space of periodic discrete signals with period N:

$$V_N = \{x : \mathbb{Z}_N \to \mathbb{C}\}.$$

An element of V_N can be viewed either as a periodic discrete function or as a vector in $\mathbb{C}^N$.

For a fixed filter $h \in \mathbb{C}^N$, circular convolution defines a linear operator

$$\mathcal{C}_h : V_N \to V_N, \qquad \mathcal{C}_h(x) = h * x.$$

For each frequency $f \in \{0, \dots, N-1\}$, we use the normalized Fourier vector ϕ_f defined in (3.1).

The fundamental result connecting circular convolution and the Fourier basis is then

$$\mathcal{C}_h(\phi_f) = \lambda_f \phi_f, \tag{3.13}$$

or, component by component,

$$(h * \phi_f)[n] = \lambda_f \phi_f[n],$$

where

$$\lambda_f = \sum_{k=0}^{N-1} h[k] e^{-2i\pi fk/N}.$$

Note that λ_f depends on the filter h and the frequency f, but not on the position n. Thus, when the input to the circular convolution operator is a Fourier mode, convolution does not alter its spatial form: it simply multiplies it by a complex scalar. In other words, λ_f is an eigenvalue and ϕ_f an eigenfunction of the circular convolution operator $\mathcal{C}_h$.

Proof.

$$\begin{aligned}
(\mathcal{C}_h(\phi_f))[n] &= (h * \phi_f)[n] \\
&= \sum_{k=0}^{N-1} h[k]\, \phi_f[(n-k) \bmod N] \\
\text{(periodicity of } \phi_f) \quad &= \sum_{k=0}^{N-1} h[k]\, \phi_f[n-k] \\
\text{(property of exponentials)} \quad &= \sum_{k=0}^{N-1} h[k] e^{-2\mathrm{i}\pi fk/N} \phi_f[n] \\
&= \left(\sum_{k=0}^{N-1} h[k] e^{-2\mathrm{i}\pi fk/N} \right) \phi_f[n] \\
&= \lambda_f \phi_f[n].
\end{aligned}$$

□

This result corresponds to the discrete convolution theorem: in the Fourier domain, circular convolution becomes frequency-wise multiplication. Thus, when the input is a pure Fourier mode ϕ_f, convolution does not change its frequency; it only multiplies it by a scalar. We therefore obtain Equation 3.13.

3.4.1 Consequence: Diagonalization of Circulant Matrices

For a fixed filter h, 1D circular convolution defines a linear map $\mathbb{C}^N \to \mathbb{C}^N$, since for all $x, y \in \mathbb{C}^N$ and all $\alpha, \beta \in \mathbb{C}$,

$$h * (\alpha x + \beta y) = \alpha(h * x) + \beta(h * y)$$

Proof.

$$\begin{aligned}
(h * (\alpha x + \beta y))[n] &= \sum_{k=0}^{N-1} h[k]\, (\alpha x + \beta y)[(n-k) \bmod N] \\
&= \sum_{k=0}^{N-1} h[k]\, (\alpha x[(n-k) \bmod N] + \beta y[(n-k) \bmod N]) \\
&= \alpha \sum_{k=0}^{N-1} h[k]\, x[(n-k) \bmod N] + \beta \sum_{k=0}^{N-1} h[k]\, y[(n-k) \bmod N] \\
&= \alpha(h * x)[n] + \beta(h * y)[n].
\end{aligned}$$

□

Since this operation is linear, the result reviewed in Appendix A guarantees that, once a basis has been chosen, it can be represented by a matrix $C_h \in \mathbb{C}^{N \times N}$ such that, for every input signal $x \in \mathbb{C}^N$,

$$y = C_h x.$$

Here, C_h is a circulant matrix constructed from the filter h. Its first row is arranged to reproduce $y[0]$, and each subsequent row is obtained by a circular shift:

$$C_h = \begin{pmatrix} h[0] & h[N-1] & h[N-2] & \cdots & h[1] \\ h[1] & h[0] & h[N-1] & \cdots & h[2] \\ h[2] & h[1] & h[0] & \cdots & h[3] \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ h[N-1] & h[N-2] & h[N-3] & \cdots & h[0] \end{pmatrix}.$$

Indeed, $y[0] = h[0]x[0] + h[1]x[N-1] + \cdots + h[N-1]x[1]$ is the first row of C_h multiplied by the vector x. Applying the same reasoning iteratively gives the other elements of y.

In the standard basis of $\mathbb{C}^N$, the linear operator $\mathcal{C}_h$ is represented by the circulant matrix C_h. The relation $\mathcal{C}_h(\phi_f) = \lambda_f \phi_f$ can then be rewritten in matrix form:

$$C_h \phi_f = \lambda_f \phi_f \tag{3.14}$$

Thus, each Fourier function ϕ_f is an eigenvector of the circulant matrix C_h, with associated eigenvalue λ_f. This means that, in the circular setting, convolution acts diagonally in the Fourier basis.

Using the Fourier matrix F_N defined in (3.2), we collect the eigenvalues in the diagonal matrix

$$\Lambda_h = \operatorname{diag}(\lambda_0, \lambda_1, \ldots, \lambda_{N-1}).$$

Combining the vector equalities (3.14) for all frequencies gives

$$\begin{aligned} & & C_h F_N &= F_N \Lambda_h \\ \text{(by unitarity of } F_N) \qquad & \iff & C_h &= F_N \Lambda_h F_N^H. \end{aligned} \tag{3.15}$$

3.5 Circular Convolution in Dimension 2

We now extend the preceding result to square images. Let $X \in \mathbb{C}^{N \times N}$ be an image and $h \in \mathbb{C}^{N \times N}$ a filter. 2D circular convolution is defined by

$$Y[m,n] = (h * X)[m,n] = \sum_{a=0}^{N-1} \sum_{b=0}^{N-1} h[a,b]\, X[(m-a) \bmod N,\ (n-b) \bmod N]. \tag{3.16}$$

We use the 2D Fourier images defined in (3.7):

$$\Phi_{u,v}[m,n] = \phi_u[m]\phi_v[n].$$

As in dimension 1, a circular shift of a Fourier basis function does not change its form, but multiplies it by a complex factor:

$$\phi_u[(m-a) \bmod N] = e^{-2\pi iua/N}\phi_u[m], \qquad \phi_v[(n-b) \bmod N] = e^{-2\pi ivb/N}\phi_v[n].$$

Thus,

$$\Phi_{u,v}[(m-a) \bmod N, (n-b) \bmod N] = e^{-2\pi i(ua+vb)/N}\Phi_{u,v}[m,n],$$

where $(ua+vb) \in \mathbb{Z}_N$.

Applying 2D circular convolution to $\Phi_{u,v}$ gives

$$\begin{aligned}(h * \Phi_{u,v})[m,n] &= \sum_{a=0}^{N-1}\sum_{b=0}^{N-1} h[a,b]\,\Phi_{u,v}[(m-a) \bmod N, (n-b) \bmod N] \\ &= \left(\sum_{a=0}^{N-1}\sum_{b=0}^{N-1} h[a,b]e^{-2\pi i(ua+vb)/N}\right)\Phi_{u,v}[m,n]. \end{aligned} \tag{3.17}$$

We therefore define

$$\lambda_{u,v} = \sum_{a=0}^{N-1}\sum_{b=0}^{N-1} h[a,b]e^{-2\pi i(ua+vb)/N}. \tag{3.18}$$

Finally, we obtain

$$h * \Phi_{u,v} = \lambda_{u,v}\Phi_{u,v}. \tag{3.19}$$

3.6 Matrix Representation of a 2D Convolution

We now use the column-wise vectorization defined in (3.8). Since circular convolution is linear, there exists a matrix

$$M \in \mathbb{C}^{N^2 \times N^2}$$

such that

$$\operatorname{vec}(Y) = M \operatorname{vec}(X). \tag{3.20}$$

In other words, the coefficient of M at row $m+Nn$ and column $r+Ns$ indicates how pixel $X[r,s]$ contributes to pixel $Y[m,n]$.

The matrix M applies the filter h to every pixel of our image X in a single operation. To do so, M has a block-circulant structure with circulant blocks (BCCB), as illustrated below:

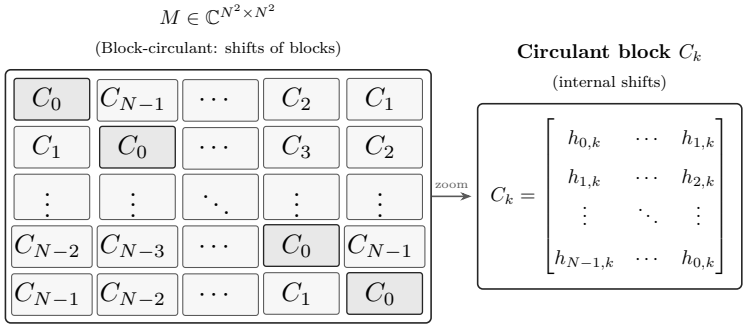


Figure 3.3: Compact structure of a BCCB (doubly circulant) matrix.

We therefore call M a **BCCB** matrix (*Block-Circulant with Circulant Blocks* [7]). Details are provided in Appendix C.2.

The spectral result obtained for 2D circular convolution then becomes, in vectorized form,

$$M \operatorname{vec}(\Phi_{u,v}) = \lambda_{u,v} \operatorname{vec}(\Phi_{u,v}).$$

3.6.1 Diagonalization by the Kronecker Product of Fourier Bases

From the preceding result 3.19, since $q_{u,v} = \operatorname{vec}(\Phi_{u,v})$, we obtain

$$M q_{u,v} = \lambda_{u,v} q_{u,v}.$$

Combining the equalities for all frequencies gives the matrix identity

$$\begin{aligned} & MQ_N = Q_N \Lambda_h \\ \iff & \text{(unitarity of } Q_N\text{)} \quad M = Q_N \Lambda_h Q_N^H. \end{aligned} \tag{3.21}$$

Here, Λ_h is the diagonal matrix containing the eigenvalues $\lambda_{u,v}$ in the same order as the columns $q_{u,v}$ of Q_N.

3.7 Extension to the Multichannel Case

So far, we have considered single-channel convolution, where the input and output images each have a single channel (for example, a grayscale image). We now consider the more general case of multichannel convolution:

$$X \in \mathbb{C}^{m_{in} \times N \times N}, \qquad Y \in \mathbb{C}^{m_{out} \times N \times N},$$

where m_{in} and m_{out} denote the number of input and output channels, respectively.

For each output channel Y_i,

$$Y_i = \sum_{j=1}^{m_{in}} h_{i,j} * X_j \qquad \text{for } i = 1, \dots, m_{out}, \tag{3.22}$$

where $h_{i,j}$ is the convolution filter applied to input channel j to produce output channel i.

3.7.1 Channel Mixing at a Fixed Frequency

Consider a spatial frequency (u, v) and the corresponding Fourier image $\Phi_{u,v}$. Assume that each input channel contains this same spatial frequency, but with a different amplitude:

$$X_j = \alpha_j \Phi_{u,v}, \qquad j = 1, \dots, m_{\text{in}}.$$

The vector

$$\alpha = (\alpha_1, \dots, \alpha_{m_{\text{in}}})^T$$

therefore contains the amplitudes of this frequency in the different input channels.

Substituting these inputs into (3.22) and using linearity of convolution gives, for output channel i,

$$\begin{aligned} Y_i &= \sum_{j=1}^{m_{\text{in}}} h_{i,j} * X_j \\ &= \sum_{j=1}^{m_{\text{in}}} \alpha_j (h_{i,j} * \Phi_{u,v}) \\ &= \sum_{j=1}^{m_{\text{in}}} \alpha_j \lambda_{u,v}^{i,j} \Phi_{u,v} \\ &= \left(\sum_{j=1}^{m_{\text{in}}} \lambda_{u,v}^{i,j} \alpha_j \right) \Phi_{u,v}. \end{aligned}$$

We then introduce the mixing matrix associated with frequency (u, v):

$$G^{(u,v)} \in \mathbb{C}^{m_{\text{out}} \times m_{\text{in}}}, \qquad G_{i,j}^{(u,v)} = \lambda_{u,v}^{i,j}.$$

If $\beta \in \mathbb{C}^{m_{\text{out}}}$ denotes the output amplitudes of this same frequency, we can simply write

$$\beta = G^{(u,v)} \alpha.$$

This expression summarizes the role of multichannel convolution: for a fixed spatial frequency, it performs linear mixing across channels.

3.7.2 Global Matrix Form

To express the multichannel operation in global matrix form, we define

$$\mathbf{x} = \begin{pmatrix} \text{vec}(X_1) \\ \vdots \\ \text{vec}(X_{m_{\text{in}}}) \end{pmatrix} \in \mathbb{C}^{m_{\text{in}} N^2}, \qquad \mathbf{y} = \begin{pmatrix} \text{vec}(Y_1) \\ \vdots \\ \text{vec}(Y_{m_{\text{out}}}) \end{pmatrix} \in \mathbb{C}^{m_{\text{out}} N^2}.$$

For each channel pair (i, j), let $B^{i,j}$ denote the spatial convolution matrix associated with filter $h_{i,j}$, that is, the matrix applying convolution between input channel j and output channel i. We use the vectorized 2D Fourier basis Q_N defined in (3.11). For each channel pair (i, j), the matrix $B^{i,j}$ can therefore be diagonalized as

$$B^{i,j} = Q_N D^{i,j} Q_N^H,$$

where

$$D^{i,j} = \text{diag}(\lambda_{u,v}^{i,j})_{(u,v) \in \{0,\dots,N-1\}^2}.$$

The diagonal entries of $D^{i,j}$ are ordered according to the flattened index $w(u, v)$ defined in (3.12).

We can now introduce the global matrix $M \in \mathbb{C}^{m_{out} N^2 \times m_{in} N^2}$, which represents the entire

convolutional layer as a block matrix of size $m_{out} \times m_{in}$, where each block (i, j) corresponds to the application of filter $h_{i,j}$:

$$M = \begin{pmatrix} B^{1,1} & \dots & B^{1,m_{in}} \\ \vdots & \ddots & \vdots \\ B^{m_{out},1} & \dots & B^{m_{out},m_{in}} \end{pmatrix}.$$

Since the same basis Q_N diagonalizes all spatial blocks $B^{i,j}$, we can factor out the spatial change of basis on either side of the block matrix. This gives

$$M = (I_{m_{out}} \otimes Q_N) L (I_{m_{in}} \otimes Q_N)^H \tag{3.23}$$

where $L \in \mathbb{C}^{m_{out}N^2 \times m_{in}N^2}$ is a block matrix with diagonal blocks:

$$L = \begin{pmatrix} D^{1,1} & \dots & D^{1,m_{in}} \\ \vdots & \ddots & \vdots \\ D^{m_{out},1} & \dots & D^{m_{out},m_{in}} \end{pmatrix}.$$

Proof. By the definition of the Kronecker product, the operations $I_{m_{out}} \otimes Q_N$ and $(I_{m_{in}} \otimes Q_N)^H = I_{m_{in}} \otimes Q_N^H$ generate block-diagonal matrices:

$$I_{m_{out}} \otimes Q_N = \begin{pmatrix} Q_N & 0 & \cdots & 0 \\ 0 & Q_N & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & Q_N \end{pmatrix}, \quad I_{m_{in}} \otimes Q_N^H = \begin{pmatrix} Q_N^H & 0 & \cdots & 0 \\ 0 & Q_N^H & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & Q_N^H \end{pmatrix}.$$

Evaluating the block matrix product on the left applies Q_N to each block in the corresponding rows of L:

$$(I_{m_{out}} \otimes Q_N) L = \begin{pmatrix} Q_N D^{1,1} & \dots & Q_N D^{1,m_{in}} \\ \vdots & \ddots & \vdots \\ Q_N D^{m_{out},1} & \dots & Q_N D^{m_{out},m_{in}} \end{pmatrix}.$$

Multiplying this result on the right by the second block-diagonal matrix applies Q_N^H to each block in the corresponding columns:

$$(I_{m_{out}} \otimes Q_N) L (I_{m_{in}} \otimes Q_N)^H = \begin{pmatrix} Q_N D^{1,1} Q_N^H & \cdots & Q_N D^{1,m_{in}} Q_N^H \\ \vdots & \ddots & \vdots \\ Q_N D^{m_{out},1} Q_N^H & \cdots & Q_N D^{m_{out},m_{in}} Q_N^H \end{pmatrix}.$$

Since the single-channel convolution decomposition gives $B^{i,j} = Q_N D^{i,j} Q_N^H$ for every pair (i, j), we obtain

$$(I_{m_{out}} \otimes Q_N) L (I_{m_{in}} \otimes Q_N)^H = \begin{pmatrix} B^{1,1} & \cdots & B^{1,m_{in}} \\ \vdots & \ddots & \vdots \\ B^{m_{out},1} & \cdots & B^{m_{out},m_{in}} \end{pmatrix} = M.$$

□

This recovers the form of **Proposition 1** in the paper by Tsuzuku and Sato [14]. It separates two phenomena:

- the matrices $(I \otimes Q_N)$ perform the spatial change of basis to the Fourier domain;
- the central matrix L contains the channel mixing, organized frequency by frequency.

3.8 Linear Stacking of Convolutions

We now introduce the notion of stacked convolutional layers. The aim is to mathematically model the operation of a deep convolutional neural network, which consists of several layers applied successively.

As shown in the preceding sections, our linear framework allows us to represent each convolutional layer with stride 1 (stride denotes the step size; see C.2 for an illustration) by a large global matrix $M^{(k)}$. Let m_k denote the number of input channels of layer k, and m_{k+1} the number of output channels:

$$M^{(k)} : \mathbb{C}^{m_k N^2} \longrightarrow \mathbb{C}^{m_{k+1} N^2}.$$

Thus, for a vectorized input $X \in \mathbb{C}^{m_1 N^2}$, the output of the first layer is

$$X_1 = M^{(1)} X \in \mathbb{C}^{m_2 N^2}.$$

Applying a second convolutional layer gives

$$X_2 = M^{(2)} X_1 = M^{(2)} (M^{(1)} X) = (M^{(2)} M^{(1)}) X.$$

Continuing in this way for d successive convolutional layers gives

$$M(X) = \left(M^{(d)} M^{(d-1)} \cdots M^{(1)} \right) X. \tag{3.24}$$

3.8.1 Simplification by Telescoping Transformations

The main advantage of the matrix decomposition in Equation (3.23) lies in its behavior when several layers are multiplied. Consider two successive layers $M^{(1)}$ and $M^{(2)}$. Their product is

$$\begin{aligned} M^{(2)}M^{(1)} &= \left[(I_{m_3} \otimes Q_N)L^{(2)}(I_{m_2} \otimes Q_N)^H\right]\left[(I_{m_2} \otimes Q_N)L^{(1)}(I_{m_1} \otimes Q_N)^H\right] \\ &= (I_{m_3} \otimes Q_N)L^{(2)} \underbrace{\Big((I_{m_2} \otimes Q_N)^H(I_{m_2} \otimes Q_N)\Big)}_{I_{m_2N^2}} L^{(1)}(I_{m_1} \otimes Q_N)^H \end{aligned}$$

By induction, for a network of d layers, the global matrix M simplifies to

$$M = (I_{m_{d+1}} \otimes Q_N)L(I_{m_1} \otimes Q_N)^H$$

where $L = L^{(d)}L^{(d-1)} \cdots L^{(1)}$ represents the accumulated channel mixing.

3.8.2 Frequency Decoupling and the Mixing Matrix $G^{(w)}$

Since each internal block of the matrices $L^{(k)}$ is diagonal, their product preserves this structure. Indeed, block (i, j) of the product of two block matrices $L^{(2)}L^{(1)}$ is

$$(L^{(2)}L^{(1)})_{i,j} = \sum_k D^{(2)}_{i,k} D^{(1)}_{k,j}.$$

The product of two diagonal matrices is diagonal, and a sum of diagonal matrices remains diagonal. Consequently, each block of the global matrix $L = L^{(d)}L^{(d-1)} \cdots L^{(1)}$ is still a diagonal matrix of size $N^2 \times N^2$.

Thus, in a linear stack of circular convolutions with stride 1, the frequency-wise structure is preserved. For a given frequency, only the distribution across channels changes.

The idea is therefore as follows: to study what the network does at a given frequency, we do not need to examine the entire large matrix L. It is sufficient to look within each block $D^{(i,j)}$.

We collect these coefficients in a small matrix

$$G^{(w)} \in \mathbb{C}^{m_{\text{out}} \times m_{\text{in}}},$$

defined by

$$\left(G^{(w)}\right)_{i,j} = \left(D^{(i,j)}\right)_{w,w}. \tag{3.25}$$

The matrix $G^{(w)}$ therefore describes the network's behavior at frequency w. It no longer describes the entire image, but only the channel mixing at that specific frequency. This allows the study of a large convolutional layer to be reduced to the study of a much smaller matrix, frequency by frequency.

3.8.3 Sensitivity Analysis Using Singular Value Decomposition (SVD)

The multichannel convolution operator can be represented by a matrix

$$M \in \mathbb{C}^{m_{\text{out}}N^2 \times m_{\text{in}}N^2}.$$

Since we seek to identify the input directions most strongly amplified by the convolutional layer, it is natural to study the singular values of this matrix.

The singular value decomposition of M is

$$M = U\Sigma V^H,$$

where U and V are unitary matrices, and Σ contains the singular values

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq 0.$$

The columns of V are called right singular vectors: they correspond to directions in input space. The columns of U are the left singular vectors: they correspond to directions in output space.

A singular value measures the amplification of an input direction. More precisely, if v_i is a right singular vector associated with σ_i and $\|v_i\|_2 = 1$, then

$$\|Mv_i\|_2 = \sigma_i.$$

Thus, large singular values correspond to input directions strongly amplified by the operator M. In particular,

$$\sigma_1(M) = \max_{x \neq 0} \frac{\|Mx\|_2}{\|x\|_2} = \max_{\|x\|_2=1} \|Mx\|_2 = \|M\|_2.$$

The direction associated with σ_1 is therefore a direction of maximum sensitivity when this singular value is simple. If it has higher multiplicity, every unit-norm direction in the associated singular subspace gives the same amplification.

In our analysis, the network's highly sensitive directions therefore correspond to right singular vectors associated with large singular values. The detailed relationship between SVD, eigenvectors of $M^H M$, and amplification is reviewed in Appendix C.6.

3.8.4 Proof: Structure of the Singular Vector Characterizing Sensitivity

Proof. Let σ be a singular value of the mixing matrix $G^{(w)} \in \mathbb{C}^{m_{\text{out}} \times m_{\text{in}}}$. Let

$$a \in \mathbb{C}^{m_{\text{in}}}, \qquad b \in \mathbb{C}^{m_{\text{out}}}$$

denote the associated right and left singular vectors. We therefore have

$$G^{(w)}a = \sigma b, \qquad G^{(w)H}b = \sigma a.$$

In the full network space, the right singular vector associated with this frequency w is

$$v = (I_{m_1} \otimes Q_N)(a \otimes e_w),$$

where e_w is the standard basis vector, equal to 1 at index w and 0 elsewhere.

We first verify this claim by applying the operator M to this candidate v:

$$\begin{aligned} Mv &= \left[(I_{m_{d+1}} \otimes Q_N)L(I_{m_1} \otimes Q_N)^H\right][(I_{m_1} \otimes Q_N)(a \otimes e_w)] \\ &= (I_{m_{d+1}} \otimes Q_N)L \underbrace{\left[(I_{m_1} \otimes Q_N)^H(I_{m_1} \otimes Q_N)\right]}_{I_{m_1 N^2}}(a \otimes e_w) \\ Mv &= (I_{m_{d+1}} \otimes Q_N)L(a \otimes e_w) \end{aligned}$$

We now examine the action of L on the vector $(a \otimes e_w)$. Recall that L is a block matrix in which each block (i, j) is the diagonal matrix $D^{(i,j)}$. The vector $(a \otimes e_w)$ can be viewed as a block vector whose j-th block is the vector $a_j e_w$. Thus, the i-th block of the resulting vector $y' = L(a \otimes e_w)$ is computed as

$$y'_i = \sum_{j=1}^{m_1} D^{(i,j)}(a_j e_w)$$

Since $D^{(i,j)}$ is diagonal, multiplying it by the standard basis vector e_w simply selects its w-th diagonal entry, multiplied by e_w. In other words,

$$D^{(i,j)}e_w = (D^{(i,j)})_{w,w}\, e_w$$

By the definition of our per-frequency mixing matrix, this coefficient $(D^{(i,j)})_{w,w}$ is exactly entry (i, j) of the matrix $G^{(w)}$. We therefore obtain directly

$$y'_i = \sum_{j=1}^{m_1}(G^{(w)})_{i,j}\, a_j\, e_w = \left(\sum_{j=1}^{m_1}(G^{(w)})_{i,j} a_j\right) e_w$$

The term in parentheses is the i-th component of the matrix product $G^{(w)}a$. Since $G^{(w)}a = \sigma b$, we obtain

$$y'_i = (\sigma b)_i e_w = \sigma b_i e_w$$

Reassembling the blocks gives

$$L(a \otimes e_w) = \sigma(b \otimes e_w). \tag{3.26}$$

We can now substitute this result back into our equation:

$$\begin{aligned} Mv &= (I_{m_{d+1}} \otimes Q_N)\,(\sigma(b \otimes e_w)) \\ &= \sigma\,(I_{m_{d+1}} b \otimes Q_N e_w) \\ Mv &= \sigma\,(b \otimes (Q_N)_w) \end{aligned}$$

This equation shows that applying M to v produces a vector of the form

$$Mv = \sigma u, \qquad u = b \otimes (Q_N)_w.$$

It remains to verify the adjoint equation. Since

$$u = (I_{m_{d+1}} \otimes Q_N)(b \otimes e_w),$$

we similarly obtain

$$M^H u = (I_{m_1} \otimes Q_N) L^H (b \otimes e_w).$$

The relation $G^{(w)H} b = \sigma a$ gives

$$L^H (b \otimes e_w) = \sigma (a \otimes e_w).$$

Thus,

$$M^H u = \sigma (a \otimes (Q_N)_w) = \sigma v.$$

Both SVD relations are therefore satisfied:

$$Mv = \sigma u, \qquad M^H u = \sigma v.$$

□

3.8.5 Verification of Orthonormality

Proof. For the set of vectors $v = a \otimes (Q_N)_w$ to rigorously form the unitary matrix V in the SVD of M, it must constitute an orthonormal basis. This follows from the mixed-product property for inner products ($(A \otimes B)^H (C \otimes D) = (A^H C) \otimes (B^H D)$):

- **Unit Norm:** Since a and $(Q_N)_w$ are unit vectors, $v^H v = (a^H a) \otimes ((Q_N)_w^H (Q_N)_w) = 1 \times 1 = 1$.
- **Orthogonality Within a Frequency:** For a given frequency w, the original orthogonality of two right singular vectors a and $\tilde{a}$ is preserved: $v^H \tilde{v} = (a^H \tilde{a}) \otimes 1 = 0$.
- **Orthogonality Across Frequencies:** For $w \neq \tilde{w}$, the Fourier columns are orthogonal: $v^H \tilde{v} = (a^H \tilde{a}) \otimes ((Q_N)_w^H (Q_N)_{\tilde{w}}) = 0$.

□

3.8.6 Final Expression for the Fourier Pattern

Finally, since $Q_N e_w = (Q_N)_w$, the singular vector expression becomes

$$v = a \otimes (Q_N)_w.$$

If index w corresponds to the frequency pair (u, v), then, using the notation $q_{u,v}$ established in Section 3.2, we can write

$$v = a \otimes q_{u,v} = a \otimes \big((F_N)_v \otimes (F_N)_u\big). \tag{3.27}$$

In other words, the singular vector consists of two parts: a vector a, which describes channel mixing, and a Fourier direction $q_{u,v}$, which describes the spatial pattern.

3.9 Skip Connections and Normalization Layers

The preceding results were established in a simplified linear setting: the layers are convolutions with stride 1, without nonlinear activations and with circular *padding*. In this setting, we showed that directions of maximum sensitivity can be expressed as a channel vector multiplied by a spatial Fourier direction, as in (3.27).

We must nevertheless verify that this conclusion remains valid when two common mechanisms of modern convolutional architectures are added: *skip connections* and normalization layers. This is the subject of Propositions 3 and 4 of Tsuzuku and Sato [14].

3.9.1 What Is a *Skip Connection*?

Modern convolutional architectures, such as residual networks (*ResNet*) [4], frequently use *skip connections*. The principle is to add the input of a block directly to its output, so that the block learns a correction to the identity rather than an entire transformation.

If $\mathcal{F}$ denotes the transformation performed by the block, its output is

$$x_{\ell+1} = \mathcal{F}(x_\ell) + x_\ell. \tag{3.28}$$

In the simple case where input and output have the same dimension, the shortcut branch is therefore the identity. Within our linear analysis, if the main block is modeled by M, the entire residual block is represented by

$$M_{\text{skip}} = M + I, \tag{3.29}$$

where I denotes the identity.

This expression is particularly useful for our spectral analysis because it shows that adding a *skip connection* does not change the nature of the underlying operator, but simply adds an identity contribution to the convolutional block.

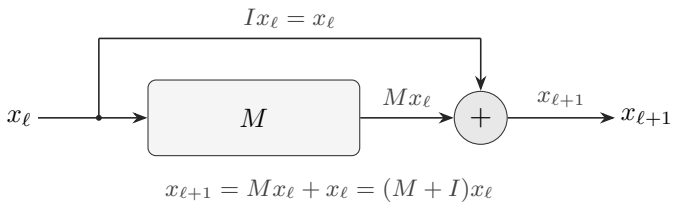


Figure 3.4: Linear interpretation of a *skip connection*. The main path applies the operator M, while the shortcut branch applies the identity. The residual block is therefore represented by the operator $M + I$.

3.9.2 The Case of a Skip Connection

Proof. We first analyze an "ideal" residual connection, where the input X and the output of the convolutional block $M(X)$ (3.24) have exactly the same dimensions (the same spatial resolution and number of channels). The overall transformation is then simply

$$M_{\text{skip}}(X) = (M^{(d)}M^{(d-1)} \dots M^{(1)})(X) + X.$$

In matrix form, this addition amounts to adding the identity matrix to our underlying operator, bringing us directly back to Equation (3.29).

In practice, when the channel dimension changes, direct addition becomes impossible and the shortcut branch instead uses a projection. If this projection is itself convolutional, it falls within the same spectral framework as the layers studied previously. The reasoning therefore remains the same, with the identity replaced by this projection.

From the factorization previously obtained in Equation (3.23), the operator M can be written as

$$M = (I_m \otimes Q_N)L(I_m \otimes Q_N)^H,$$

Since Q_N is unitary, we also have

$$I_{mN^2} = (I_m \otimes Q_N)I_{mN^2}(I_m \otimes Q_N)^H.$$

Thus,

$$\begin{aligned} M_{\text{skip}} &= M + I_{mN^2} \\ &= (I_m \otimes Q_N)L(I_m \otimes Q_N)^H + (I_m \otimes Q_N)I_{mN^2}(I_m \otimes Q_N)^H \\ &= (I_m \otimes Q_N)(L + I_{mN^2})(I_m \otimes Q_N)^H. \end{aligned}$$

The skip connection therefore does not change the spatial diagonalization basis. It only modifies the central matrix L by adding the identity.

To make this observation more explicit, fix a spatial frequency w, corresponding to a frequency pair (u, v). As explained in Section 3.8, the network's action at this frequency is described by a mixing matrix defined in (3.25),

$$G^{(w)} \in \mathbb{C}^{m \times m}.$$

Adding the identity does not change the frequency under consideration: it simply adds a direct contribution to the same channel and frequency. The mixing matrix therefore becomes

$$G_{\text{skip}}^{(w)} = G^{(w)} + I_m.$$

At a fixed frequency, the skip connection therefore simply adds a direct contribution to the mixing matrix.

We can then apply exactly the same reasoning as in Section 3.8. In particular, if a is a right singular vector of $G_{\text{skip}}^{(w)}$, the corresponding right singular vector in the full space retains the form

$$v = a \otimes (Q_N)_w,$$

that is,

$$v = a \otimes q_{u,v} = a \otimes \big((F_N)_v \otimes (F_N)_u\big),$$

A detailed algebraic verification of this claim is provided in Appendix C.3.

The skip connection therefore preserves the form of the preceding argument: for each frequency w, it simply replaces $G^{(w)}$ with $G^{(w)} + I_m$. The singular directions retain the form

$$a \otimes q_{u,v}.$$

This corresponds to Proposition 3 of Tsuzuku and Sato [14].

□

3.9.3 The Case of Normalization Layers

Proof. Modern convolutional architectures also use normalization layers, particularly *batch normalization*, introduced by Ioffe and Szegedy [6]. At first glance, these layers may appear to fall outside the preceding framework because they are not convolutions. However, during testing (also called *inference*), the statistics used by batch normalization are fixed: the mean and variance are no longer computed from the current minibatch, but come from statistics learned during training. This stabilization turns a complex operation into a fixed affine transformation.

For an output channel i, it applies normalization followed by a learned scaling and shift. At inference, the statistics used are no longer those of the current minibatch, but estimates fixed during training. This gives

$$\mathrm{BN}_i(Y_i) = \gamma_i \frac{Y_i - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}} + \beta_i,$$

where μ_i and σ_i^2 are the fixed statistics of channel i, while γ_i and β_i are learned parameters.

Since μ_i, σ_i^2, γ_i, and β_i are constant at inference, this transformation is affine. Defining

$$s_i = \frac{\gamma_i}{\sqrt{\sigma_i^2 + \varepsilon}}, \qquad t_i = \beta_i - s_i \mu_i,$$

gives

$$\mathrm{BN}_i(Y_i) = s_i Y_i + t_i.$$

First, suppose the convolution is written without a bias, in keeping with the linear framework used so far in (3.22). At inference, batch normalization then acts channel by channel as an affine transformation, giving

$$\mathrm{BN}_i(Y_i) = s_i \sum_{j=1}^{m_{\text{in}}} h_{i,j} * X_j + t_i = \sum_{j=1}^{m_{\text{in}}} (s_i h_{i,j}) * X_j + t_i.$$

The linear part is therefore still a convolution, with modified filters

$$h'_{i,j} = s_i h_{i,j}.$$

The constant term t_i is an affine bias added to the output channel. Since spectral analysis concerns the linear part of the operator, this constant term does not modify the singular directions under study.

Thus, a convolution followed by batch normalization at inference can be represented as a new convolution with modified weights and biases.

However, this conclusion applies to inference mode. During training, batch normalization depends on the statistics of the current minibatch. The operator is then no longer exactly the same for each individual input. This recovers the result of Proposition 4 of Tsuzuku and Sato [14]. □

3.9.4 Conclusion

Propositions 3 and 4 show that skip connections and normalization at inference still fit within the linear framework studied here. They modify the mixing matrices or the effective convolution parameters, but not the general form of the singular directions obtained previously.

3.10 Downsampling Layers: Stride and Average Pooling

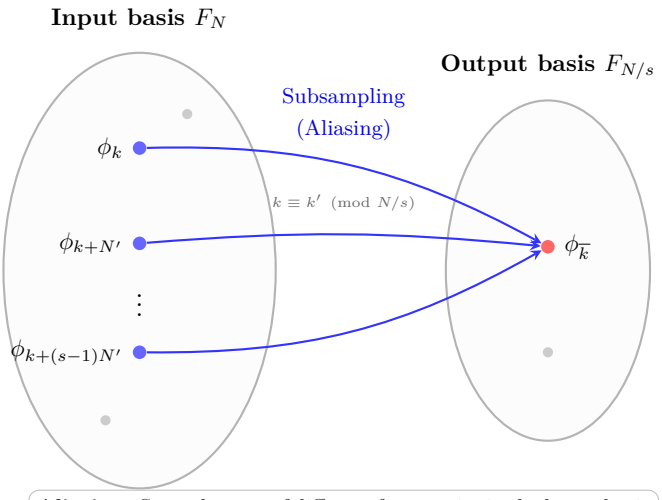
This section revisits Proposition 5 of Tsuzuku and Sato [14]. The goal is to better understand what happens when a layer reduces the spatial size of an image. In particular, we examine why, in this case, the singular directions no longer necessarily correspond to a single Fourier frequency.

What is a *stride*? Before proceeding, we define the notion of *stride*. In a standard convolutional layer, the filter slides over the image one position at a time: this is referred to as a *stride* of 1.

However, the filter can instead be required to make larger jumps, moving by s positions at each step, where s is a strictly positive integer. This parameter s is the *stride*. The immediate effect of $s > 1$ is to reduce the spatial resolution of the output, since a value is no longer computed at every position, but only every s positions, as illustrated in Figure C.2.

From a frequency perspective, this reduction in resolution has an essential consequence: it makes some frequencies indistinguishable. When only one position in s is retained, several distinct Fourier components in input space can produce the same component in output space. In other words, *stride* introduces spectral folding (*aliasing* [2]), as illustrated in Figure 3.5: different

frequencies before downsampling map to the same frequency afterward. This mechanism plays a central role in the frequency analysis of downsampling layers.



Aliasing: Several waves of different frequencies in the larger basis F_N (spaced by $N' = N/s$) are projected onto a single wave in the reduced basis $F_{N/s}$. They therefore become indistinguishable after downsampling.

Figure 3.5: Set-based representation of generalized spectral folding. The arrows illustrate the loss of injectivity: a set of s source frequencies contributes to a single output frequency.

Why the previous results no longer apply directly. The previous results relied on the fact that circular convolution with stride 1 preserves spatial size and acts separately on each Fourier frequency. This separation yields singular vectors associated with a single frequency. Once a layer reduces spatial resolution, this property disappears: subsampling groups several input frequencies into the same output frequency. We now formalize this grouping, or aliasing.

3.10.1 Spatial Subsampling and Frequency Aliasing

To simplify the analysis, we first isolate the effect of subsampling, independently of any convolution. Assume that s divides N, and let

$$N' = \frac{N}{s}.$$

We define the subsampling operator

$$P_s : \mathbb{C}^N \longrightarrow \mathbb{C}^{N'}$$

by

$$(P_s x)_r = x_{sr}, \qquad r = 0, \dots, N' - 1.$$

In other words, P_s retains one value every s positions.

Now let $(F_N)_k$ be a 1D Fourier direction of size N. Applying P_s to this direction gives, for

every $r = 0, \dots, N' - 1$,

$$\begin{aligned}\left(P_s(F_N)_k\right)_r &= ((F_N)_k)_{sr} \\ &= \frac{1}{\sqrt{N}} \exp\left(2\pi i \frac{ksr}{N}\right) \\ &= \frac{1}{\sqrt{N}} \exp\left(2\pi i \frac{kr}{N'}\right).\end{aligned}$$

The preceding exponential depends on k modulo N'. Writing

$$\overline{k} = k \bmod N',$$

and since $N = sN'$, so that

$$\frac{1}{\sqrt{N}} = \frac{1}{\sqrt{s}} \frac{1}{\sqrt{N'}},$$

we obtain

$$P_s(F_N)_k = \frac{1}{\sqrt{s}} (F_{N'})_{\overline{k}}. \tag{3.30}$$

This means that, after subsampling, frequency k at size N becomes frequency $\overline{k}$ at size N'. This equation precisely expresses *aliasing*. Two frequencies k and k' such that

$$k \equiv k' \pmod{N'}$$

become identical after subsampling. Thus, for each frequency $\overline{k} \in \{0, \dots, N' - 1\}$, there is a set of s source frequencies in the original signal,

$$\mathcal{A}_k = \left\{\overline{k} + qN' \mid q \in \{0, \dots, s - 1\}\right\},$$

that become indistinguishable after subsampling.

In other words, the subsampling operator is not **injective** in the frequency domain: s distinct frequencies in the domain, spaced by N', project onto a single frequency in the codomain.

Extension to the two-dimensional case. We now extend this reasoning to images of size $N \times N$. We define the 2D spatial subsampling operator

$$P_s^{(2D)} : \mathbb{C}^{N \times N} \longrightarrow \mathbb{C}^{N' \times N'}$$

by

$$\left(P_s^{(2D)} X\right)_{r,t} = X_{sr,st}, \qquad r, t = 0, \dots, N' - 1. \tag{3.31}$$

It therefore retains one row in s and one column in s.

After vectorization, let Π_s denote the matrix associated with the operator $P_s^{(2D)}$. For the vectorized Fourier direction

$$q_{u,v} = (F_N)_v \otimes (F_N)_u,$$

the 1D result can be applied along each of the two dimensions. Using the 1D result obtained in

(3.30) along each dimension gives

$$\begin{aligned}\Pi_s q_{u,v} &= \Pi_s\big((F_N)_v \otimes (F_N)_u\big)\\ &= (P_s(F_N)_v) \otimes (P_s(F_N)_u)\\ &= \left(\frac{1}{\sqrt{s}}(F_{N'})_{\overline{v}}\right) \otimes \left(\frac{1}{\sqrt{s}}(F_{N'})_{\overline{u}}\right)\\ &= \frac{1}{s}(F_{N'})_{\overline{v}} \otimes (F_{N'})_{\overline{u}}. \end{aligned} \tag{3.32}$$

Here,

$$\overline{u} = u \bmod N', \qquad \overline{v} = v \bmod N'.$$

Thus, two 2D frequencies (u, v) and (u', v') become indistinguishable after subsampling whenever

$$u \equiv u' \pmod{N'}, \qquad v \equiv v' \pmod{N'}.$$

For each output frequency $(\overline{u}, \overline{v})$, the set of input frequencies that project onto it is therefore

$$\mathcal{A}_{\overline{u},\overline{v}} = \left\{(\overline{u} + q_1 N',\, \overline{v} + q_2 N') \mid q_1, q_2 \in \{0, \dots, s-1\}\right\}. \tag{3.33}$$

3.10.2 Strided Convolution and Spectral Folding

We now return to a convolutional layer. A convolution with stride s can be viewed as an ordinary convolution with stride 1, followed by noncommutative spatial subsampling. After vectorization, if M_h denotes the operator associated with circular convolution with stride 1 and kernel h, and Π_s denotes the subsampling matrix associated with $P_s^{(2D)}$, the convolution operator with stride s is

$$M_h^{(s)} = \Pi_s M_h. \tag{3.34}$$

This expression means that all convolution values are computed first, and only positions whose two spatial indices are multiples of s are then retained.

Since M_h is a circular convolution with stride 1, this direction is an eigendirection of the operator. Using the diagonalization recalled in (3.21), we have

$$M_h q_{u,v} = \lambda_{u,v} q_{u,v},$$

where $\lambda_{u,v}$ is the eigenvalue associated with frequency (u, v).

Applying subsampling next, and using (3.32), gives

$$\begin{aligned} M_h^{(s)} q_{u,v} &= \Pi_s M_h q_{u,v}\\ &= \lambda_{u,v} \Pi_s q_{u,v}\\ &= \frac{\lambda_{u,v}}{s} q_{\overline{u},\overline{v}}^{(N')}, \end{aligned} \tag{3.35}$$

where

$$q_{\overline{u},\overline{v}}^{(N')} = (F_{N'})_{\overline{v}} \otimes (F_{N'})_{\overline{u}}.$$

Here,

$$\overline{u} = u \bmod N', \qquad \overline{v} = v \bmod N'.$$

Equation (3.35) shows that strided convolution preserves part of the Fourier structure, but with an essential difference: several input frequencies can be mapped to the same output frequency. More precisely, all frequencies belonging to the same aliasing class

$$\mathcal{A}_{\overline{u},\overline{v}} = \{(\overline{u} + q_1 N', \overline{v} + q_2 N') \mid q_1, q_2 \in \{0, \dots, s-1\}\}$$

are associated with the same output frequency $(\overline{u}, \overline{v})$.

To express this grouping in matrix form, we introduce the matrix

$$\mathcal{S}_s \in \mathbb{C}^{N'^2 \times N^2},$$

which describes spectral folding in the Fourier basis. Its columns correspond to input frequencies (u, v) and its rows to output frequencies (α, β). We define

$$(\mathcal{S}_s)_{(\alpha,\beta),(u,v)} = \begin{cases} \frac{1}{s}, & \text{if } \alpha = u \bmod N' \text{ and } \beta = v \bmod N', \\ 0, & \text{otherwise.} \end{cases} \tag{3.36}$$

Thus, for a given input frequency (u, v), the matrix $\mathcal{S}_s$ places a single nonzero coefficient in the row corresponding to the output frequency

$$(u \bmod N', v \bmod N').$$

This coefficient equals $\frac{1}{s}$, as in (3.32).

The action of subsampling on the entire Fourier basis can then be written as

$$\Pi_s Q_N = Q_{N'} \mathcal{S}_s. \tag{3.37}$$

Since Q_N is unitary, we obtain the factorization

$$\Pi_s = Q_{N'} \mathcal{S}_s Q_N^H. \tag{3.38}$$

This expression confirms that $\mathcal{S}_s$ represents aliasing in the frequency domain. Indeed, if

$$\widehat{x} = Q_N^H x$$

denotes the Fourier coefficients of the input, the Fourier coefficients of the subsampled output are given by

$$\widehat{y} = Q_{N'}^H \Pi_s x = \mathcal{S}_s \widehat{x}.$$

By the definition of $\mathcal{S}_s$, this gives, for each output frequency (α, β),

$$\widehat{y}_{\alpha,\beta} = \frac{1}{s} \sum_{(u,v) \in \mathcal{A}_{\alpha,\beta}} \widehat{x}_{u,v}. \tag{3.39}$$

In other words, subsampling does not retain the Fourier coefficients separately: it sums the coefficients belonging to the same aliasing class. This mechanism explains why, in a strided layer, singular directions are no longer necessarily associated with a single Fourier frequency.

3.10.3 Composition with a Convolution of Stride 1

We can now combine subsampling with circular convolution of stride 1. Consider a multichannel convolutional layer with stride 1, represented by the matrix operator M. From the factorization established in (3.23),

$$M = (I_{m_{\text{out}}} \otimes Q_N) L (I_{m_{\text{in}}} \otimes Q_N)^H,$$

A convolution with stride s is then obtained by first applying convolution with stride 1, and then retaining only one position in s along each spatial direction. Since this reduction is performed separately on each output channel, we introduce

$$\Pi_s^{\text{out}} = I_{m_{\text{out}}} \otimes \Pi_s.$$

The operator corresponding to convolution with stride s is then

$$M^{(s)} = \Pi_s^{\text{out}} M.$$

Substituting the Fourier factorization of M and the factorization of Π_s obtained in (3.38) gives

$$\begin{aligned} M^{(s)} &= (I_{m_{\text{out}}} \otimes \Pi_s)(I_{m_{\text{out}}} \otimes Q_N) L (I_{m_{\text{in}}} \otimes Q_N)^H \\ &= (I_{m_{\text{out}}} \otimes Q_{N'})(I_{m_{\text{out}}} \otimes \mathcal{S}_s) L (I_{m_{\text{in}}} \otimes Q_N)^H. \end{aligned}$$

This expression makes the difference from the stride-1 case clear. The factor L still acts frequency by frequency. The factor $\mathcal{S}_s$, however, subsequently groups several input frequencies into a single output frequency.

To examine this grouping more closely, we use the same reasoning as above. Let

$$\widehat{x}_{u,v} \in \mathbb{C}^{m_{\text{in}}} \qquad \text{and} \qquad \widehat{z}_{u,v} = G^{(u,v)} \widehat{x}_{u,v}.$$

We then apply subsampling for a reduced frequency (α, β):

$$\begin{aligned} \widehat{y}_{\alpha,\beta} &= \frac{1}{s} \sum_{(u,v) \in \mathcal{A}_{\alpha,\beta}} \widehat{z}_{u,v} \\ &= \frac{1}{s} \sum_{(u,v) \in \mathcal{A}_{\alpha,\beta}} G^{(u,v)} \widehat{x}_{u,v}. \end{aligned} \tag{3.40}$$

To express this relation in matrix form, we fix an arbitrary ordering of the frequencies in the aliasing class:

$$\mathcal{A}_{\alpha,\beta} = \{(u_1, v_1), \ldots, (u_{s^2}, v_{s^2})\}.$$

We then collect the corresponding input coefficients in a single vector:

$$\widehat{x}_{\mathcal{A}_{\alpha,\beta}} = \begin{pmatrix} \widehat{x}_{u_1,v_1} \\ \vdots \\ \widehat{x}_{u_{s^2},v_{s^2}} \end{pmatrix} \in \mathbb{C}^{m_{\text{in}}s^2}.$$

This gives the matrix

$$G_s^{(\alpha,\beta)} = \frac{1}{s}\begin{pmatrix} G^{(u_1,v_1)} & G^{(u_2,v_2)} & \dots & G^{(u_{s^2},v_{s^2})} \end{pmatrix} \in \mathbb{C}^{m_{\text{out}}\times m_{\text{in}}s^2}. \tag{3.41}$$

This matrix is simply a compact way to express the sum in (3.40). Multiplying $G_s^{(\alpha,\beta)}$ by the stacked vector $\widehat{x}_{\mathcal{A}_{\alpha,\beta}}$ gives exactly

$$\frac{1}{s}\sum_{(u,v)\in\mathcal{A}_{\alpha,\beta}} G^{(u,v)}\widehat{x}_{u,v}.$$

By construction, therefore,

$$\widehat{y}_{\alpha,\beta} = G_s^{(\alpha,\beta)}\widehat{x}_{\mathcal{A}_{\alpha,\beta}}.$$

This formulation reduces the study of the structural sensitivity of a convolution with stride s to a local analysis, one output frequency at a time. For each reduced frequency (α,β), we therefore study the singular values and vectors of the matrix $G_s^{(\alpha,\beta)}$.

3.10.4 Construction of Right Singular Vectors

Proof. To simplify the notation, let

$$\varphi_{u,v}^{(N)} = (F_N)_v \otimes (F_N)_u, \qquad \varphi_{\alpha,\beta}^{(N')} = (F_{N'})_\beta \otimes (F_{N'})_\alpha.$$

Fix an output frequency (α,β) and its aliasing class $\mathcal{A}_{\alpha,\beta}$. Choosing an arbitrary ordering of this class, we write

$$\mathcal{A}_{\alpha,\beta} = \{(u_1,v_1),\dots,(u_{s^2},v_{s^2})\}.$$

Let

$$p = \begin{pmatrix} p_{u_1,v_1} \\ \vdots \\ p_{u_{s^2},v_{s^2}} \end{pmatrix} \in \mathbb{C}^{m_{\text{in}}s^2}, \qquad p_{u_i,v_i} \in \mathbb{C}^{m_{\text{in}}},$$

be a right singular vector of $G_s^{(\alpha,\beta)}$, associated with a singular value σ and a left singular vector $q \in \mathbb{C}^{m_{\text{out}}}$. We therefore have

$$G_s^{(\alpha,\beta)}p = \sigma q, \qquad G_s^{(\alpha,\beta)H}q = \sigma p. \tag{3.42}$$

From the blocks $p_{u,v}$, we construct the input vector

$$r = \sum_{(u,v)\in\mathcal{A}_{\alpha,\beta}} p_{u,v} \otimes \varphi_{u,v}^{(N)}. \tag{3.43}$$

This vector is therefore no longer associated with a single Fourier frequency: it is a combination of all frequencies belonging to the same aliasing class.

We now show that r is a right singular vector of $M^{(s)}$. By (3.40), the action of $M^{(s)}$ on the Fourier coefficients associated with the class $\mathcal{A}_{\alpha,\beta}$ is given by

$$\widehat{y}_{\alpha,\beta} = \frac{1}{s} \sum_{(u,v)\in\mathcal{A}_{\alpha,\beta}} G^{(u,v)} p_{u,v}.$$

By the definition of $G_s^{(\alpha,\beta)}$, this sum is simply

$$\widehat{y}_{\alpha,\beta} = G_s^{(\alpha,\beta)} p.$$

Using the singular-vector relation (3.42), we therefore obtain

$$\widehat{y}_{\alpha,\beta} = \sigma q.$$

In the spatial domain, this gives

$$M^{(s)} r = \sigma q \otimes \varphi_{\alpha,\beta}^{(N')}.$$

Defining

$$w = q \otimes \varphi_{\alpha,\beta}^{(N')},$$

gives

$$M^{(s)} r = \sigma w. \tag{3.44}$$

It remains to verify the adjoint equation. The adjoint of $M^{(s)}$ maps an output frequency (α, β) back to all input frequencies in the class $\mathcal{A}_{\alpha,\beta}$. This gives

$$M^{(s)H} w = \sum_{(u,v)\in\mathcal{A}_{\alpha,\beta}} \frac{1}{s} \left(G^{(u,v)H} q \right) \otimes \varphi_{u,v}^{(N)}.$$

The relation

$$G_s^{(\alpha,\beta)H} q = \sigma p$$

implies, block by block, that

$$\frac{1}{s} G^{(u,v)H} q = \sigma p_{u,v}, \qquad (u,v) \in \mathcal{A}_{\alpha,\beta}.$$

Substituting into the preceding expression gives directly

$$M^{(s)H} w = \sigma r. \tag{3.45}$$

The two relations

$$M^{(s)}r = \sigma w, \qquad M^{(s)H}w = \sigma r$$

therefore show that r and w form a pair of right and left singular vectors of $M^{(s)}$.

Finally, normalization is preserved. The frequencies in $\mathcal{A}_{\alpha,\beta}$ are distinct, so the corresponding directions $\varphi_{u,v}^{(N)}$ are orthonormal. Thus,

$$\|r\|_2^2 = \sum_{(u,v)\in\mathcal{A}_{\alpha,\beta}} \|p_{u,v}\|_2^2 = \|p\|_2^2.$$

If p is normalized, then r is also normalized. Similarly, if q is normalized, then $w = q \otimes \varphi_{\alpha,\beta}^{(N')}$ is normalized.

This recovers the result stated in Proposition 5 of Tsuzuku and Sato [14]: in a downsampling layer, singular directions are no longer necessarily pure Fourier directions. They can be combinations of frequencies belonging to the same aliasing class.

□

3.10.5 Average Pooling as a Special Case

Average pooling replaces each local window of size $s \times s$ with the average of its pixels, as illustrated in Figure C.3. This operation therefore reduces spatial resolution, like a strided convolution. The difference is that a fixed averaging filter is applied before subsampling.

In the circular setting used here, this filter can be represented by the kernel

$$h_{\text{avg}}[a,b] = \begin{cases} \dfrac{1}{s^2}, & \text{if } a,b \in \{0,\dots,s-1\}, \\ 0, & \text{otherwise,} \end{cases}$$

where spatial indices are interpreted modulo N. Convolution with h_{avg} therefore computes a local average, after which subsampling retains one position in s along each spatial direction.

In the multichannel case, this operation is applied separately to each channel. If m denotes the number of channels and $C_{h_{\text{avg}}} \in \mathbb{C}^{N^2\times N^2}$ denotes the circular convolution matrix associated with the averaging kernel, the *average pooling* operator is

$$M_{\text{avg}}^{(s)} = (I_m \otimes \Pi_s)(I_m \otimes C_{h_{\text{avg}}}). \tag{3.46}$$

This is exactly the case studied previously: circular convolution with stride 1, followed by subsampling. The averaging kernel acts as a low-pass filter, attenuating certain frequencies before spectral folding. However, the aliasing mechanism remains the same: several input frequencies can be grouped into a single output frequency.

Consequently, the right singular vectors of an *average pooling* layer can also be chosen as combinations of frequencies belonging to the same aliasing class.

3.11 Real-Valued Image Constraint

Proposition 6 of Tsuzuku and Sato [14] addresses the problem of applying the methods just developed to practical cases. Images processed by a network are real-valued, whereas an individual Fourier function is generally complex. The idea is to show that the requirement that an image be real-valued can be met using the Hermitian symmetry of its Fourier coefficients. To this end, we introduce the tools needed to understand this symmetry.

We begin with the 2D discrete Fourier transform [15]. For an image $X \in \mathbb{C}^{N\times N}$, we define

$$\widehat{X}_{u,v} = S(X)_{u,v} = \sum_{m=0}^{N-1}\sum_{n=0}^{N-1} X[m,n]\,\exp\left(-2\pi i\frac{um+vn}{N}\right), \qquad u,v \in \{0,\dots,N-1\}.$$

The image can be reconstructed from its Fourier coefficients using the inverse transform:

$$X[m,n] = \frac{1}{N^2}\sum_{u=0}^{N-1}\sum_{v=0}^{N-1} \widehat{X}_{u,v}\,\exp\left(2\pi i\frac{um+vn}{N}\right). \tag{3.47}$$

These formulas are compatible with the Fourier bases introduced previously, up to a normalization factor. Here, we follow the notation $S(X)$ used in the paper [14] to reproduce Proposition 6 directly.

3.11.1 Hermitian Symmetry of Fourier Coefficients

The requirement that an image be real-valued corresponds to a particular symmetry of its Fourier coefficients. More precisely, for an image $X \in \mathbb{C}^{N\times N}$,

$$X \in \mathbb{R}^{N\times N} \quad\Longleftrightarrow\quad \widehat{X}_{u,v} = \widehat{X}^*_{(-u) \bmod N,\,(-v) \bmod N}, \qquad u,v \in \{0,\dots,N-1\}. \tag{3.48}$$

This property is the Hermitian symmetry of the Fourier transform of a real-valued signal. It is a standard result in discrete signal processing [11], and here corresponds to Proposition 6 of Tsuzuku and Sato [14].

We first prove the forward implication. Assume that X is real-valued, that is,

$$X[m,n]^* = X[m,n] \qquad \text{for all } m,n.$$

By the definition of the 2D DFT,

$$\begin{aligned}
\widehat{X}^*_{u,v} &= \left(\sum_{m=0}^{N-1}\sum_{n=0}^{N-1} X[m,n]\,\exp\left(-2\pi i\frac{um+vn}{N}\right)\right)^* \\
&= \sum_{m=0}^{N-1}\sum_{n=0}^{N-1} X[m,n]^*\,\exp\left(2\pi i\frac{um+vn}{N}\right) \\
&= \sum_{m=0}^{N-1}\sum_{n=0}^{N-1} X[m,n]\,\exp\left(-2\pi i\frac{((-u)m+(-v)n)}{N}\right) \\
&= \widehat{X}_{(-u) \bmod N,\,(-v) \bmod N}.
\end{aligned}$$

We therefore obtain

$$\widehat{X}_{u,v} = \widehat{X}^*_{(-u) \bmod N, (-v) \bmod N}.$$

We now prove the converse. Suppose the Fourier coefficients satisfy the symmetry (3.48). We want to show that the reconstructed image is real-valued. Starting from the inverse Fourier transform (3.47), we compute the conjugate of $X[m, n]$:

$$\begin{aligned} X[m,n]^* &= \left(\frac{1}{N^2} \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} \widehat{X}_{u,v} \exp\left(2\pi i \frac{um+vn}{N}\right) \right)^* \\ &= \frac{1}{N^2} \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} \widehat{X}^*_{u,v} \exp\left(-2\pi i \frac{um+vn}{N}\right). \end{aligned}$$

Recall that the Hermitian symmetry assumption gives

$$\widehat{X}^*_{u,v} = \widehat{X}_{(-u) \bmod N, (-v) \bmod N}.$$

Therefore,

$$X[m,n]^* = \frac{1}{N^2} \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} \widehat{X}_{(-u) \bmod N, (-v) \bmod N} \exp\left(-2\pi i \frac{um+vn}{N}\right).$$

Changing indices according to

$$u' = (-u) \bmod N, \qquad v' = (-v) \bmod N,$$

gives

$$\begin{aligned} X[m,n]^* &= \frac{1}{N^2} \sum_{u'=0}^{N-1} \sum_{v'=0}^{N-1} \widehat{X}_{u',v'} \exp\left(2\pi i \frac{u'm+v'n}{N}\right) \\ &= X[m,n]. \end{aligned}$$

Thus, $X[m, n]^* = X[m, n]$ for all m, n, so X is real-valued.

We have therefore established the equivalence:

$$X \in \mathbb{R}^{N \times N} \quad \Longleftrightarrow \quad \widehat{X}_{u,v} = \widehat{X}^*_{(-u) \bmod N, (-v) \bmod N}.$$

3.11.2 Consequence for Fourier Perturbations

The preceding equivalence applies directly to the perturbations we wish to add to images. Since the images processed by the network are real-valued,

$$x \in \mathbb{R}^{N \times N \times c},$$

an admissible perturbation must also be real-valued in the spatial domain. Thus, for each channel, its Fourier coefficients must satisfy Hermitian symmetry:

$$\widehat{\delta}_{(-u) \bmod N,\,(-v) \bmod N} = \widehat{\delta}^{*}_{u,v}. \tag{3.49}$$

In other words, if a frequency (u, v) is used, its opposite frequency

$$((-u) \bmod N,\ (-v) \bmod N)$$

must also be used with the conjugate coefficient. This condition guarantees that the inverse Fourier transform yields a real-valued perturbation.

There are a few special cases in which a frequency equals its own opposite modulo N. These frequencies satisfy

$$(u, v) = ((-u) \bmod N,\ (-v) \bmod N).$$

In this case, the preceding condition simply requires the Fourier coefficient to be real. For even N, this applies in particular to the frequencies

$$(0, 0), \qquad (N/2, 0), \qquad (0, N/2), \qquad (N/2, N/2).$$

Outside these special cases, frequencies must be taken in conjugate pairs.

Tsuzuku and Sato use this idea in their *Single Fourier Attack* [14]. For a non-self-conjugate frequency, a *Single Fourier Attack* perturbation can then be written as

$$\delta_{u,v} = \eta \left[(1 + i)\, q_{u,v} + (1 - i)\, q_{(-u) \bmod N,\,(-v) \bmod N} \right]. \tag{3.50}$$

The coefficients $(1 + i)$ and $(1 - i)$ are conjugates of one another, and the two Fourier directions correspond to opposite frequencies. The perturbation therefore satisfies condition (3.49), guaranteeing that it is real-valued in the spatial domain.

The parameter η controls the perturbation amplitude in this theoretical expression. In practice, adversarial experiments instead impose a bound on the pixels, for example,

$$\|\delta\|_{\infty} \leq \varepsilon.$$

3.12 Limitations and Scope of the Theoretical Framework

The preceding results provide a clear mathematical explanation for the special role of Fourier directions in convolutional networks. However, the exact scope of this result must be specified.

3.12.1 An Exact Result in a Controlled Setting

The preceding reasoning is exact in a specific setting: circular convolutions, linear layers, and sensitivity analysis using singular values. Under these assumptions, the factorization in (3.23) allows the network to be studied frequency by frequency. The resulting singular directions then

have the form

$$a \otimes q_{u,v},$$

where $q_{u,v}$ describes the spatial Fourier pattern and a the channel mixing.

This result should therefore be understood as a structural property of convolutions, rather than a complete description of an actual CNN.

3.12.2 Differences from Practical Architectures

Architectures used in practice depart from this exact setting. *Zero-padding* often replaces circular padding, preventing exact diagonalization by the Fourier basis. Downsampling layers, such as strided convolutions and *average pooling*, introduce aliasing, as already discussed in Section 3.10.

Finally, ReLU activations and *max-pooling* make the network nonlinear. The complete network can therefore no longer be viewed as a single linear operator diagonalizable in the Fourier basis. However, these operations have a piecewise linear structure, allowing a local interpretation of frequency analysis to be retained.

3.12.3 Local Scope of Frequency Analysis in Nonlinear CNNs

In this subsection, f denotes the mapping performed by the network up to the logits, before the final *argmax* operation, which is not an affine map.

Networks using piecewise linear activations, such as ReLU, can be viewed as piecewise affine functions: input space is divided into regions, and on each region the network performs an affine map [9, 1]. This observation clarifies the scope of frequency analysis in practical CNNs: the complete network is not globally linear, but its behavior can be described locally by an affine map as long as the activation and pooling choices remain fixed.

This leads to the following proposition:

Proposition 3.1 (Local Linearization of a CNN with ReLU and Max-Pooling)**.** *Let f be a convolutional network consisting of convolutional layers, ReLU activations, and max-pooling layers. For any input x that does not lie on a boundary between linear regions, that is, when the ReLU pre-activations have nonzero signs and the selected maxima in the pooling windows are unique, there exists a neighborhood $\mathcal{V}_x$ of x such that, for every $z \in \mathcal{V}_x$,*

$$f(z) = A_x z + b_x,$$

where A_x is a matrix depending on the linear region containing x.

Proof. A ReLU activation acts coordinate by coordinate according to

$$\mathrm{ReLU}(t) = \max(t, 0).$$

Consider a given layer of the network, and let a_ℓ denote its pre-activation vector. If the sign of each coordinate of a_ℓ is fixed in a neighborhood of x, then ReLU acts as a linear map on this region:

$$\mathrm{ReLU}(a_\ell) = D_\ell(x) a_\ell,$$

where $D_\ell(x)$ is a diagonal matrix whose entries are 0 or 1 depending on whether the corresponding coordinates are inactive or active.

Similarly, a max-pooling layer selects the coordinate with the largest value in each local window. When this maximum is unique in every window, the selected indices remain unchanged in a sufficiently small neighborhood of x. In this region, max-pooling therefore becomes a linear selection map:

$$\mathrm{MaxPool}(z) = P_\ell(x)z,$$

where $P_\ell(x)$ is a selection matrix depending on the region under consideration. This matrix encodes both the selection of local maxima and the spatial reduction induced by pooling.

Thus, as long as we remain in a region where the ReLU signs and the indices selected by max-pooling do not change, every network operation is affine or linear. The composition of these operations is therefore itself affine. There consequently exist a matrix A_x and a vector b_x such that

$$f(z) = A_x z + b_x.$$

This local region can be understood as the set of inputs for which the ReLU activation masks and the indices selected by the max-pooling layers remain unchanged. [1]

□

This proposition shows that a CNN consisting of convolutions, ReLU, and max-pooling is piecewise affine. Around an input x, as long as the perturbation changes neither the activation masks nor the positions selected by pooling, the network's behavior is exactly linear up to a bias. Thus, for every $x + \delta \in \mathcal{V}_x$,

$$f(x + \delta) - f(x) = J_f(x)\delta.$$

In such a region, the Jacobian can be viewed as a product of matrices associated with the different network operations. For example, in the schematic case of a stack of convolution–ReLU–pooling blocks, we can write

$$J_f(x) = W_L P_{L-1}(x) D_{L-1}(x) M_{L-1} \cdots P_1(x) D_1(x) M_1,$$

where the matrices M_ℓ represent convolutions, the matrices $D_\ell(x)$ the ReLU masks, the matrices $P_\ell(x)$ the selections induced by max-pooling, and W_L the final linear part.

The matrices $D_\ell(x)$ and $P_\ell(x)$ depend on the input. In general, they therefore prevent global diagonalization of the network in the Fourier basis. A Fourier direction is no longer an exact eigenvector of the complete mapping.

However, the convolution operators M_ℓ remain present in the Jacobian product. Frequency analysis therefore retains a local scope: it no longer describes the global network exactly, but it can still explain why certain frequencies are more strongly amplified than others around particular inputs.

[1] A more detailed study could explicitly define this region, for example as the set of points (z) with the same ReLU pre-activation signs and indices of maxima in the pooling windows as the input (x). Analyzing the size of this region would provide a more precise measure of the local validity of the linearization, but lies beyond the scope of this work.

The *Single Fourier Attack* can then be interpreted as empirically exploiting this local sensitivity. This interpretation extends the linearity hypothesis of Goodfellow et al. [3].

3.12.4 Practical Scope of the Result

The theoretical framework does not predict exactly which frequencies will be most sensitive for a given architecture. This sensitivity also depends on the learned weights, the dataset, padding, normalization, nonlinearities, and downsampling layers.

The analysis does, however, explain why Fourier directions are natural candidates: they arise directly in the linear study of convolutions. Even in a nonlinear CNN, this structure may remain partially visible through the model's local Jacobian. The experiments will therefore assess the extent to which this intuition remains observable in practice.

4 Experimental Analysis of CNN Frequency Sensitivity

4.1 Chapter Objective and Scope

In the previous chapter, we showed that Fourier functions play a special role in the analysis of convolution operators. In the idealized linear setting, certain frequency directions emerge as preferred directions of the convolutional operator. In a real CNN, which includes ReLU and max-pooling, this property is no longer global or exact, but it can still be interpreted locally through the model's Jacobian.

The objective of this chapter is therefore to test experimentally whether this frequency sensitivity remains observable in a CNN trained on real data. We do not aim to reproduce all the experiments of Tsuzuku and Sato [14], but to adopt their central mechanism: systematically testing perturbations constructed from Fourier frequencies and measuring their ability to change the model's predictions.

Our protocol focuses on `VGGSmallBN`, a VGG-style architecture with batch normalization, trained on CIFAR-10 and CIFAR-100. For each frequency (u, v), we construct a universal Fourier perturbation and then measure the proportion of predictions that change. This measure, called the *fool ratio*, allows us to construct a frequency sensitivity map of the model.

Our reproduction is deliberately more limited than that of Tsuzuku and Sato, but retains the essential elements: constructing a Fourier perturbation, controlling its amplitude, calculating the *fool ratio*, and generating a frequency heatmap.

The main amplitude is set to

$$\epsilon = 10/255,$$

as in the CIFAR experiments in the original paper. We then add two experimental controls: a comparison with random noise of the same amplitude and a sweep over several values of ϵ.

Finally, unlike the paper's heatmap algorithm, which evaluates each frequency on a minibatch, we evaluate each frequency on the full test set. This makes the computation more expensive but provides a more stable estimate of the *fool ratio*.

4.2 Experimental Setup

4.2.1 Datasets

We use CIFAR-10 and CIFAR-100. Both datasets contain RGB images of size 32×32, allowing us to retain the same frequency domain in both experiments:

$$(u, v) \in \{0, \dots, 31\}^2.$$

CIFAR-10 contains 10 classes and provides a relatively stable initial setting for reproduction. CIFAR-100 contains 100 classes at the same image resolution; the task is therefore more difficult,

since the categories are more numerous and often visually similar. Comparing the two allows us to observe whether frequency sensitivity persists as classification becomes more fine-grained.

4.2.2 Architecture

For this experimental reproduction, we use a VGG-style convolutional architecture with batch normalization, which we call `VGGSmallBN`. The aim is not to reproduce a historical VGG architecture exactly [12], but to retain its general principle: stacking several simple convolutional blocks consisting of 3×3 convolutions, batch normalization, ReLU activations, and spatial reduction layers.

The network consists of four convolutional blocks. Each block contains two 3×3 convolutions, each followed by batch normalization and a ReLU activation. The block ends with *max-pooling* of size 2×2, which halves the spatial resolution. The classification head then applies *adaptive average pooling*, flattens the resulting tensor, and uses a final linear layer. This architecture is summarized in Figure 4.1.

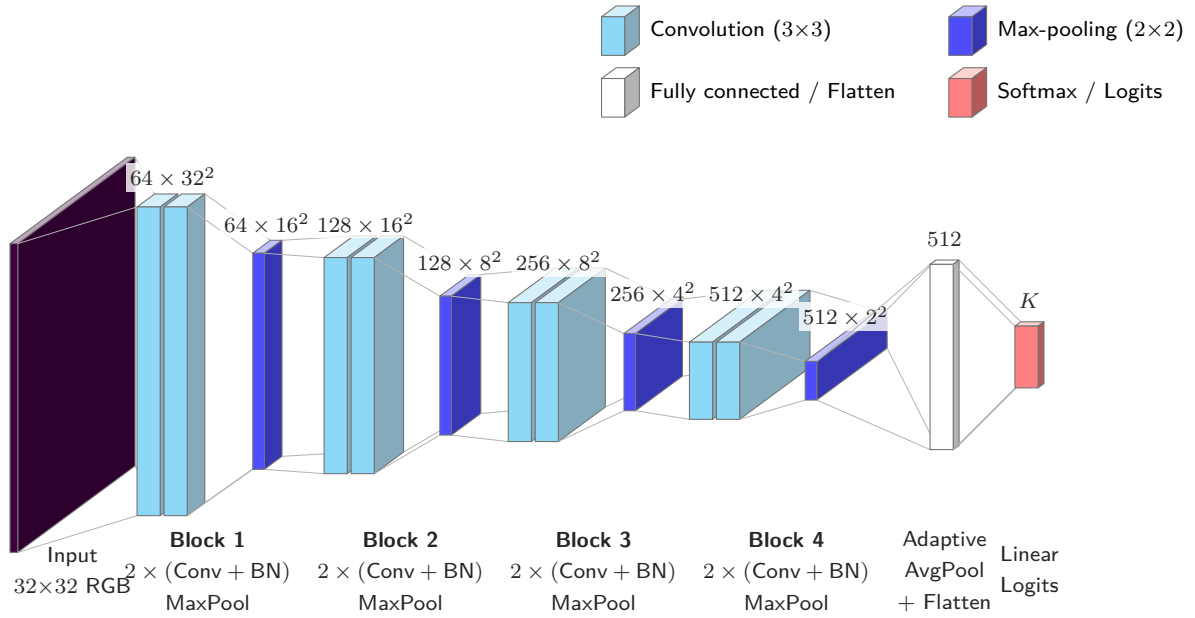


Figure 4.1: Illustration of the `VGGSmallBN` architecture used, where K denotes the number of classes in the dataset: $K = 10$ for CIFAR-10 and $K = 100$ for CIFAR-100.

4.2.3 Training Parameters

Parameter	Selected value
Number of epochs	100
Optimizer	SGD
Initial learning rate	0.1
Momentum	0.9
Weight decay	$5 \cdot 10^{-4}$
Scheduler	Cosine annealing
Batch size	256
Datasets	CIFAR-10 and CIFAR-100
Architecture	`VGGSmallBN`

Table 4.1: Training parameters used for the `VGGSmallBN` models.

Momentum stabilizes weight updates and accelerates optimization. *Weight decay* acts as a regularizer: it penalizes excessively large weights, thereby limiting the risk of overfitting.

The initial learning rate is set to 0.1 and then gradually reduced using a *cosine annealing* scheduler. The idea is to let the model learn quickly at the beginning of training and then make finer adjustments toward the end. Finally, we train the model for 100 epochs with a batch size of 256, allowing efficient use of the available GPU while maintaining stable training.

4.2.4 Baseline Performance

Dataset	Number of classes	Validation	Test
CIFAR-10	10	93.86 %	93.07 %
CIFAR-100	100	73.04 %	72.55 %

Table 4.2: Baseline performance of the `VGGSmallBN` models before perturbations are added.

The difference between the two datasets is expected: CIFAR-100 contains ten times as many classes as CIFAR-10, with categories that are often more visually similar. In both cases, performance is high enough for the perturbation study to be interpretable: the models are not perfect, but they serve as credible baseline classifiers.

4.3 Constructing Fourier Perturbations

The general idea is to construct a universal perturbation. Unlike a conventional adversarial attack, the perturbation is not optimized separately for each image. It depends only on a frequency (u, v) and an amplitude ϵ. We then examine whether this frequency direction, although it is not

a global eigenvector of the real network, is sufficient to change the model's predictions on a large number of images.

4.3.1 Single Fourier Attack

The *Single Fourier Attack* (SFA) proposed by Tsuzuku and Sato [14] constructs a perturbation from a single Fourier frequency. The main idea is simple: instead of optimizing a different perturbation for each image, we choose a frequency (u, v), construct the corresponding spatial pattern, and then add that same pattern to all images.

In our protocol, a frequency is represented by a pair

$$(u, v) \in \{0, \dots, N-1\}^2,$$

where N denotes the spatial size of the image.

For a given frequency (u, v), let $\delta_{u,v}$ denote the perturbation constructed from that frequency. The perturbed image is then defined by

$$\tilde{x}_i = \mathrm{clip}_{[0,1]}\left(x_i + \delta_{u,v,i}\right) = \begin{cases} 0, & \text{if } x_i + \delta_{u,v,i} < 0, \\ x_i + \delta_{u,v,i}, & \text{if } x_i + \delta_{u,v,i} \in [0,1], \\ 1, & \text{if } x_i + \delta_{u,v,i} > 1. \end{cases} \tag{4.1}$$

Here, x is the original image, $\tilde{x}$ is the perturbed image, and clipping ensures that pixel values remain within the valid interval $[0, 1]$.

In practice, for each frequency (u, v), we first construct a real-valued spatial pattern $p_{u,v}$. This pattern is then normalized in the ℓ_∞ norm so that its maximum amplitude equals the chosen amplitude ϵ:

$$\delta_{u,v} = \epsilon \frac{p_{u,v}}{\|p_{u,v}\|_\infty}. \tag{4.2}$$

Thus, if one frequency achieves a higher *fool ratio* than another, this is not because it was tested with a larger perturbation, but because its frequency direction disrupts the model more.

For the RGB images in CIFAR-10 and CIFAR-100, the same spatial pattern is applied to all three color channels. The final perturbation therefore has the same frequency structure in each channel. After the perturbation is added, the image is clipped to the interval $[0, 1]$ and normalized using the statistics of the dataset before being passed to the model.

Algorithm 1 summarizes the procedure used in our implementation.

Algorithm 1 Single Fourier Attack used in our protocol

input: image x, frequency (u, v), amplitude ϵ

output: perturbed image $\tilde{x}$

Construct the real-valued spatial pattern $p_{u,v}$

Normalize this pattern in the ℓ_∞ norm:

$$\delta_{u,v} \leftarrow \epsilon \frac{p_{u,v}}{\|p_{u,v}\|_\infty}$$

Add the perturbation to the image:

$$x_{\text{pert}} \leftarrow x + \delta_{u,v}$$

Apply clipping:

$$\tilde{x} \leftarrow \text{clip}_{[0,1]}(x_{\text{pert}})$$

return $\tilde{x}$

4.3.2 Amplitude Control

In our main protocol, we use the same bound as in the original paper for CIFAR:

$$\epsilon = \frac{10}{255}.$$

This value means that each pixel can be changed by at most 10 levels on a scale from 0 to 255.

For each frequency (u, v), the perturbation is normalized to satisfy

$$\|\delta_{u,v}\|_\infty \leq \epsilon.$$

After the perturbation is added, the image is clipped to the valid interval $[0, 1]$, as in Equation (4.1).

As an example, Figure 4.2 shows the spatial pattern obtained for frequency $(16, 24)$, which corresponds to the best direction found on CIFAR-10.

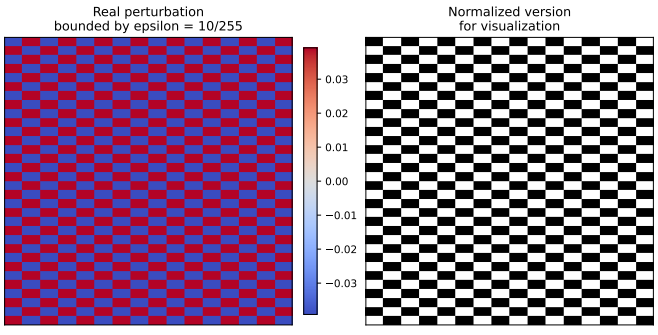


Figure 4.2: Spatial pattern associated with the SFA perturbation at frequency $(16, 24)$. The visualization is normalized to make the pattern's structure visible.

4.4 Measuring Frequency Sensitivity

Once the Fourier perturbations have been defined, their effect on the model must be measured.

4.4.1 Definition of the Fool Ratio

Let f be the model under study, and let $\mathcal{D} = \{x_1, \ldots, x_M\}$ be a set of images. For a given frequency (u, v), let $\tilde{x}_k^{(u,v)}$ denote the image obtained by adding to x_k the Fourier perturbation associated with that frequency.

The *fool ratio* measures the proportion of images for which the model's prediction changes after perturbation:

$$\mathrm{FR}(u, v) = \frac{1}{M} \sum_{k=1}^{M} \mathbf{1}\left[f(x_k) \neq f\left(\tilde{x}_k^{(u,v)}\right)\right].$$

This measure does not directly compare the prediction with the true label. It compares only the model's prediction before and after perturbation. Thus, the attack counts as successful on an image when the perturbation changes the model's decision.

The *fool ratio* therefore indicates how effectively a frequency direction can change predictions, regardless of the images' true class.

4.4.2 Generating Heatmaps

A frequency sensitivity heatmap is a matrix $H \in \mathbb{R}^{32\times32}$ in which each entry corresponds to a frequency (u, v). The value $H_{u,v}$ is the *fool ratio* obtained when the Fourier perturbation associated with that frequency is added to the test images.

The computation has three steps: first, we compute the model's predictions on clean images; next, for each frequency, we apply the corresponding perturbation; finally, we measure the proportion of predictions that change. This procedure is summarized in Algorithm 2.

Algorithm 2 Generating a frequency sensitivity heatmap

input: model f, test images $\mathcal{D}$, amplitude ϵ
output: heatmap $H \in \mathbb{R}^{32\times32}$

$y \leftarrow f(\mathcal{D})$ ▷ predictions on clean images
for $u = 0$ **to** 31 **do**
  for $v = 0$ **to** 31 **do**
    $\delta_{u,v} \leftarrow$ construct the Fourier perturbation at frequency (u, v)
    $\tilde{\mathcal{D}}_{u,v} \leftarrow \mathrm{clip}_{[0,1]}(\mathcal{D} + \delta_{u,v})$
    $\tilde{y}_{u,v} \leftarrow f(\tilde{\mathcal{D}}_{u,v})$
    $H_{u,v} \leftarrow \mathrm{FoolRatio}(y, \tilde{y}_{u,v})$
  end for
end for

4.4.3 Selecting the Most Sensitive Frequency

Once the heatmap has been computed, we select the frequency that maximizes the *fool ratio*:

$$(u^\star, v^\star) = \arg\max_{(u,v)} \mathrm{FR}(u, v).$$

This frequency corresponds to the most effective Fourier perturbation among all those tested in the protocol.

This selection is performed separately for CIFAR-10 and CIFAR-100. The resulting frequencies and their interpretation are presented in Section 4.5.2.

4.5 Experimental Results

4.5.1 Frequency Sensitivity Heatmaps

The methodology described above allows us to construct frequency sensitivity maps for the `VGGSmallBN` model. Figure 4.3 compares the results obtained on our two datasets at the same perturbation amplitude, set to $\epsilon = 10/255$.

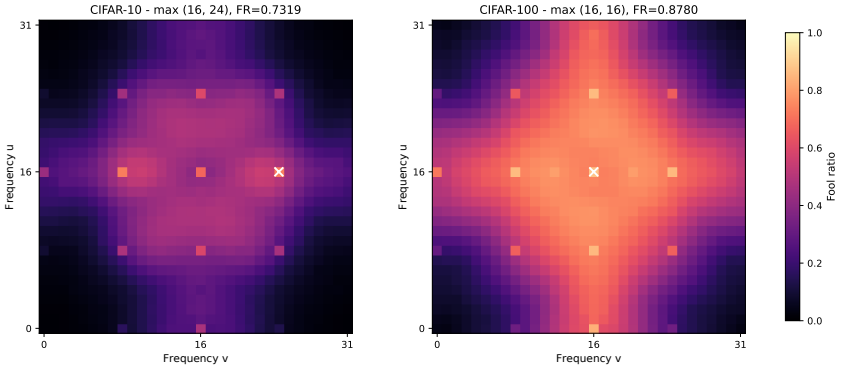


Figure 4.3: Frequency sensitivity heatmaps for `VGGSmallBN` on CIFAR-10 and CIFAR-100.

The main observation is that sensitivity is not uniform in the Fourier domain. Some frequencies cause very few prediction changes, while others produce a much higher *fool ratio*.

Both heatmaps exhibit a structured organization, with regions of high sensitivity around certain frequency lines and intersections, particularly near coordinates associated with 16. This structure suggests that the model's vulnerability is not a random effect, but depends on certain preferred frequency directions.

A descriptive comparison with the mean spectrum of clean images appears in Appendix C.7. This analysis aims to offer another way of interpreting the results through the frequency statistics of the raw data.

To complement the heatmaps, Figure 4.4 shows the distribution of *fool ratios* over the $32 \times 32 = 1024$ frequencies tested.

On CIFAR-10, most frequencies have a weak or moderate effect, and only a few directions achieve high *fool ratios*. The model's sensitivity is therefore real, but relatively localized in the frequency domain.

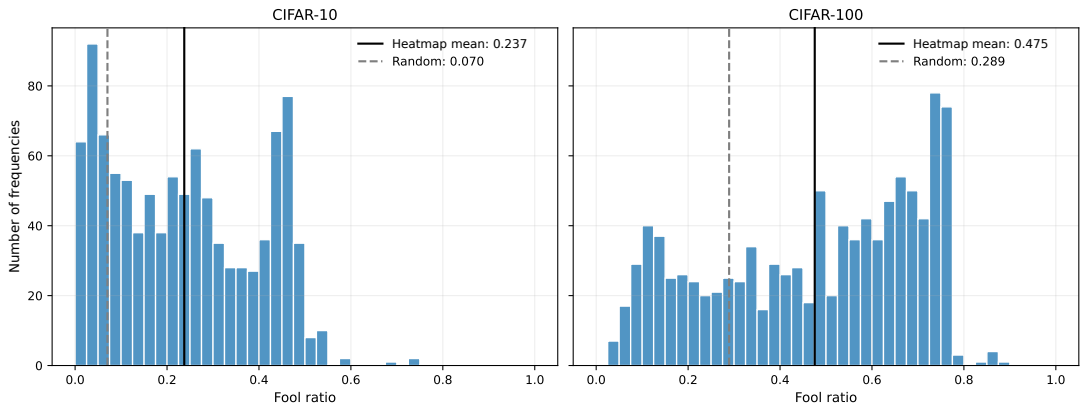


Figure 4.4: Distribution of the *fool ratios* obtained over all Fourier frequencies tested.

On CIFAR-100, the distribution shifts to the right: the mean increases from 0.2375 to 0.4755. The model is therefore more fragile overall, consistent with the greater difficulty of the task discussed in Appendix C.10. However, this fragility remains structured: not all frequencies produce the same effect, and some directions remain markedly more disruptive than others.

The histogram thus complements the visual interpretation of the heatmaps. It shows that sensitivity is not only spatially nonuniform but also quantitatively highly dispersed. A small fraction of the frequencies accounts for most of the disruptive effect.

4.5.2 Most Sensitive Frequencies

Analysis of the heatmaps identifies the most disruptive frequencies for each dataset. Table 4.3 presents the three frequencies with the highest *fool ratios*.

	CIFAR-10		CIFAR-100	
Rank	Frequency (u, v)	*Fool ratio*	Frequency (u, v)	*Fool ratio*
1	$(16, 24)$	0.7319	$(16, 16)$	0.8780
2	$(16, 8)$	0.7295	$(16, 8)$	0.8585
3	$(16, 16)$	0.6765	$(24, 16)$	0.8565

Table 4.3: Most sensitive frequencies for `VGGSmallBN` on CIFAR-10 and CIFAR-100.

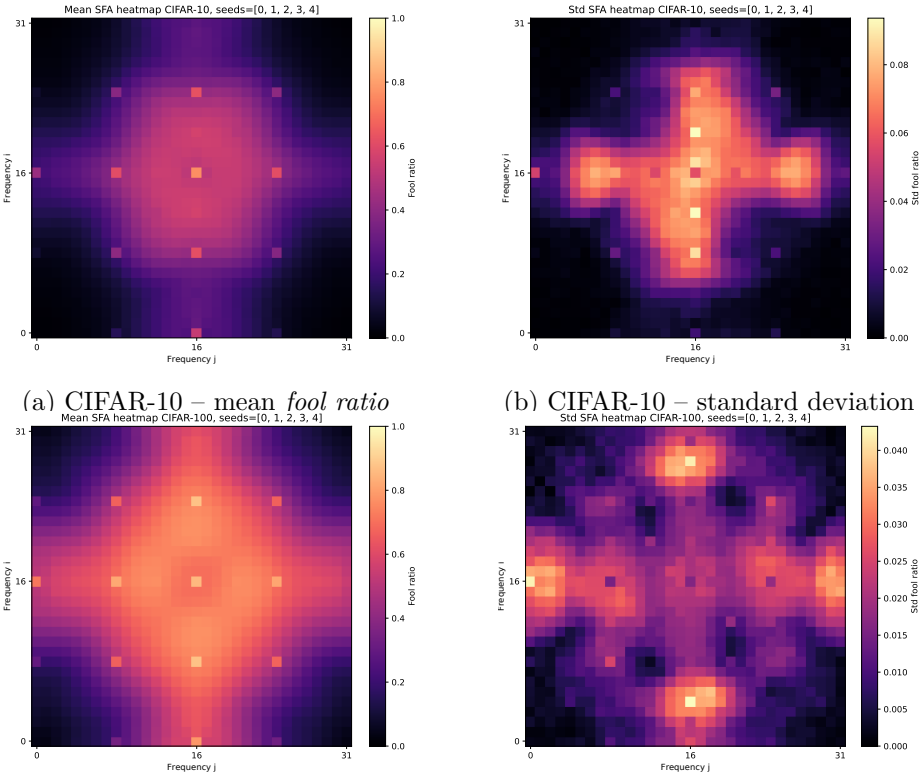
The dominant frequencies cluster around the same family of directions. On CIFAR-10, the top two frequencies, $(16, 24)$ and $(16, 8)$, are conjugates modulo 32 and, as expected, yield almost identical *fool ratios*.

On CIFAR-100, the most sensitive frequency becomes $(16, 16)$, which is its own conjugate modulo 32.

4.5.3 Stability of Sensitive Regions Across Training Runs

The preceding heatmaps show that some frequencies are much more sensitive than others. However, an important question remains: is this structure specific to a particular training run,

or does it remain observable when the initialization seed changes?



(a) CIFAR-10 – mean *fool ratio*

(b) CIFAR-10 – standard deviation

(c) CIFAR-100 – mean *fool ratio*

(d) CIFAR-100 – standard deviation

Figure 4.5: Stability of the frequency sensitivity of `VGGSmallBN` on CIFAR-10 and CIFAR-100.

To answer this question, we repeated the experiment on CIFAR-10 and CIFAR-100 with five independent training runs of `VGGSmallBN`. The training protocol remains identical for each run. Only the seed changes.

For each dataset, Figure 4.5 presents the mean heatmap obtained across the five seeds and the standard deviation associated with each frequency.

The results show that the overall heatmap structure remains highly stable across training runs. On CIFAR-10, regions of high sensitivity remain localized around the same frequency regions, particularly around the middle frequencies associated with the central axes. On CIFAR-100, this structure is even more pronounced: a broad region of high sensitivity appears around the central frequencies.

Table 4.4 summarizes the main metrics averaged across the five seeds. Clean accuracy remains close to that of the main model, whereas accuracy after the attack drops sharply. This shows that the high *fool ratio* does correspond to an actual degradation of model performance, rather than merely to prediction changes that have no effect on correct classification.

Metric	CIFAR-10	CIFAR-100
Mean clean accuracy	93.08 %	72.50 %
Mean best *fool ratio*	76.35 %	87.64 %
Mean accuracy after attack	23.39 %	11.83 %
Success rate on initially correct images	76.38 %	85.18 %

Table 4.4: Summary of the stability experiment across multiple seeds on CIFAR-10 and CIFAR-100.

4.5.4 Internal Amplification of Sensitive Frequencies

The preceding heatmaps show that some Fourier frequencies produce markedly higher *fool ratios* than others. To better understand this phenomenon, we analyzed how these perturbations propagate inside the network.

For a perturbation δ, let $G_l(\delta)$ denote the normalized amplification measured at layer l. This quantity compares the mean norm of the activation change caused by the perturbation with the norm of the input perturbation. It therefore allows us to compare the relative effect of different frequencies within the network's internal blocks and at the logit level.

To make this analysis more robust, we repeated it on the five checkpoints used in the stability experiment across multiple seeds. Frequencies are grouped using the mean heatmap: the 50 most sensitive frequencies, 50 frequencies of intermediate sensitivity, and the 50 least sensitive frequencies. The primary frequency is the best frequency on average across the five seeds.

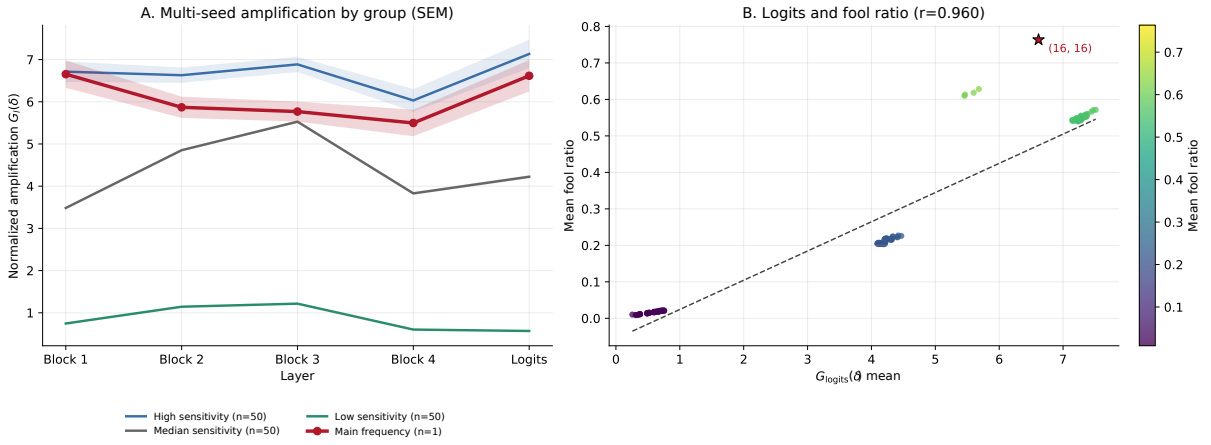


Figure 4.6: Analysis of the internal amplification of Fourier perturbations across multiple seeds in `VGGSmallBN` on CIFAR-10. Left: frequencies are grouped by their mean *fool ratio*. Bands indicate the standard error across the five seeds. Right: mean amplification at the logit level is compared with the mean *fool ratio*.

Figure 4.6 shows that, as expected, the most sensitive frequencies also produce the greatest internal amplification.

The right panel shows a strong correlation between mean amplification at the logit level and the mean *fool ratio*. This suggests that the model's final sensitivity is related not only to the

chosen pair (u, v), but also to how this perturbation is propagated and amplified by the network.

This observation connects with the theoretical framework and the local interpretation introduced in the previous chapter: although the real network is not globally linear, certain frequency directions seem to be amplified more by its local behavior, which may explain their greater effect on predictions.

A more detailed visualization of the internal heatmaps by layer is provided in Appendix C.9.

4.5.5 Comparison with Random Noise

To verify that the high values obtained by SFA are not solely due to the perturbation's amplitude, we compare the best Fourier direction with a random-noise baseline. In this baseline, each pixel is perturbed independently by uniform noise bounded within the same amplitude interval. The constraint is therefore the same as for SFA:

$$\|\delta\|_\infty \leq \epsilon, \qquad \epsilon = 10/255.$$

The difference lies in the perturbation's structure: SFA uses a coherent frequency pattern, whereas the random baseline does not favor any particular direction.

TO DO: We should ensure that $\|\delta_{SFA}\|_2 = \|\delta_{random}\|_2$

For each dataset, Figure 4.7 compares the *fool ratio* of the best SFA frequency with the mean *fool ratio* obtained using random noise.

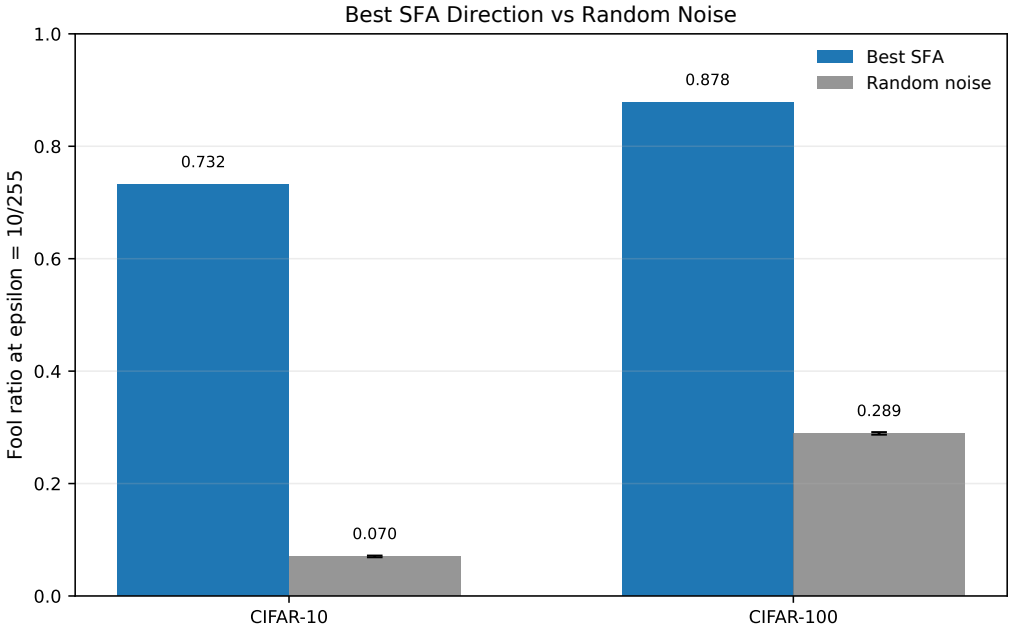


Figure 4.7: Comparison between the best SFA direction and uniform random noise of the same amplitude $\epsilon = 10/255$. For random noise, the error bar represents the standard deviation over 100 trials.

On CIFAR-10, the difference is striking: the best Fourier direction achieves a *fool ratio* of 0.7319, whereas the mean for random noise is only 0.0704. The Fourier perturbation is therefore

approximately ten times as effective as random noise of the same amplitude. This result indicates that the observed vulnerability cannot be explained solely by the size of the perturbation.

On CIFAR-100, random noise is already more disruptive, with a mean *fool ratio* of 0.2892, confirming that the model trained on CIFAR-100 is more fragile overall. However, the best SFA direction reaches 0.8780, approximately three times the mean level for random noise.

4.5.6 Effect of Perturbation Amplitude

So far, the main results have been obtained with $\epsilon = 10/255$. This value allows us to compare our results with the original protocol, but it can produce visible patterns in very low-resolution images. However, an adversarial attack aims to remain barely perceptible to humans. We therefore test whether the most sensitive frequencies retain an effect at lower values of ϵ.

To do so, we fix the most sensitive frequency for each dataset and then vary ϵ over the following set:

$$\epsilon \in \left\{ \frac{1}{255}, \frac{2}{255}, \frac{3}{255}, \frac{5}{255}, \frac{10}{255} \right\}.$$

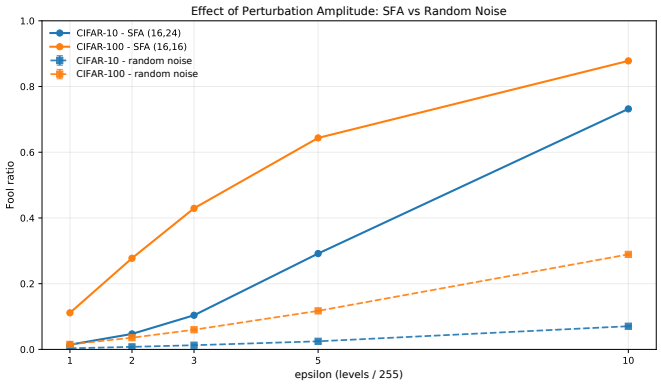


Figure 4.8: *Fool ratio* as a function of amplitude ϵ for the best SFA frequency on each dataset.

Figure 4.8 shows a steady increase in the *fool ratio* as ϵ increases. This behavior is expected: the larger the perturbation, the more it changes the input image and the more likely it is to alter the model's prediction. However, it also confirms that, in this setting, perturbation amplitude is an important factor in attack effectiveness, requiring a less subtle perturbation to obtain a more substantial effect.

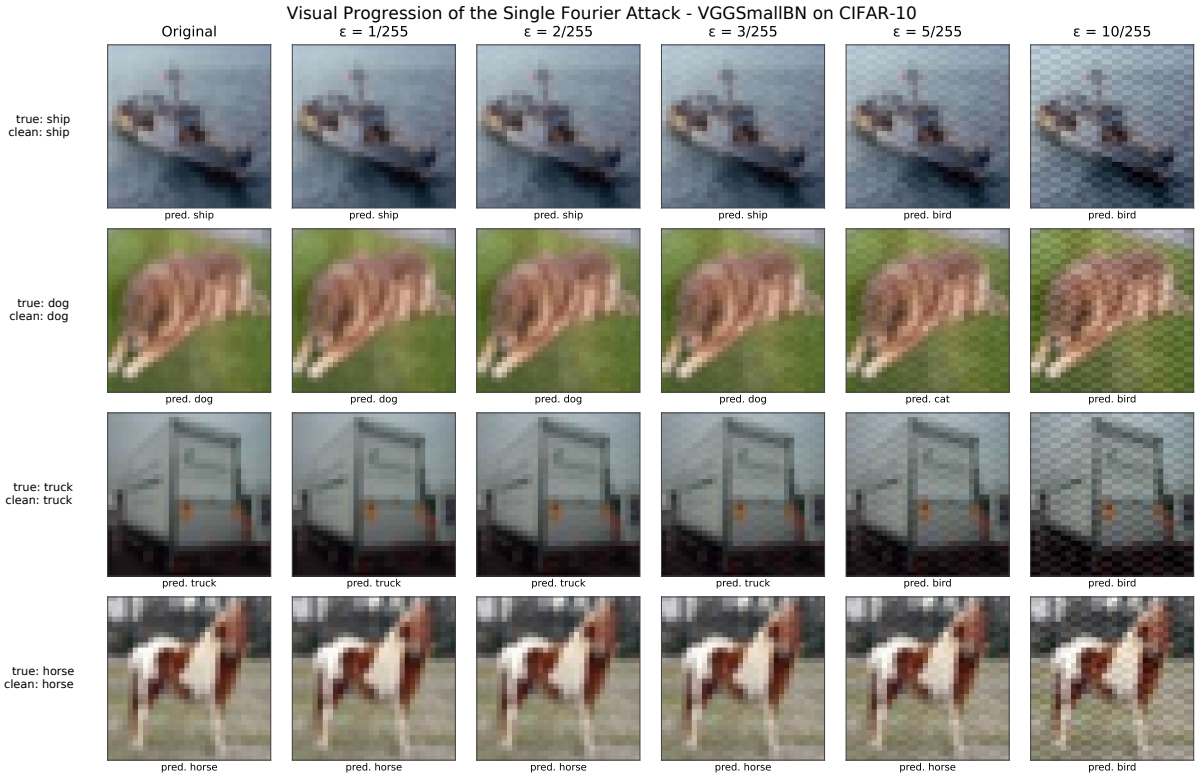


Figure 4.9: Visual progression of a *Single Fourier Attack* on CIFAR-10 images.

As Figure 4.9 illustrates, increasing the amplitude makes the Fourier pattern increasingly intrusive. In particular, from $\epsilon = 3/255$ onward, the structured grid-like perturbation begins to become clearly visible to the naked eye.

These observations should nevertheless be qualified. The experiments use very low-resolution images (32×32 pixels) and a deliberately limited architecture (`VGGSmallBN`). The conclusions should therefore not be generalized directly to high-resolution images or much deeper architectures. They do, however, show that, in our experimental setting, the effect of Fourier perturbations depends strongly on amplitude.

4.5.7 Connection with the Theoretical Framework

The experimental results in this chapter allow us to revisit the theoretical framework developed in the previous chapter. In the idealized linear model, Fourier functions emerge as preferred directions of convolution operators. In a real CNN, this property no longer applies directly to the entire network, but it may remain locally relevant through the model's Jacobian.

The experimental heatmaps support this interpretation. If all frequency directions were equivalent for the model, the sensitivity maps would be much more uniform. Yet this is not what we observe: some frequencies produce markedly higher *fool ratios* than others, even though all perturbations satisfy the same ℓ_∞ constraint.

Stability across multiple seeds reinforces this interpretation. The sensitive regions do not seem to depend solely on a particular network initialization: they remain visible when training is repeated with different seeds. The comparison with random noise also shows that the effect

arises not only from the perturbation's amplitude, but also from its frequency structure.

These results show that the intuition derived from the theoretical framework remains experimentally observable for `VGGSmallBN`: some Fourier directions are more disruptive than others, consistent with local amplification of these directions within the network.

Influence of channel mixing. Previously, we considered the possibility that a sensitive direction may also involve mixing the color channels, in the form $a \otimes \varphi_{u,v}$. The main protocol uses a simplified version of this idea: the same Fourier pattern is applied to all three channels, corresponding to a color direction of the form $(1, 1, 1)$.

A supplementary analysis of this simplification is provided in Appendix C.8. It aims to show that the choice of channel mixing has a negligible effect compared with selecting the direction that maximizes attack amplification.

4.6 Chapter Summary

The experiments confirm that the sensitivity of `VGGSmallBN` to Fourier perturbations is highly nonuniform. On both CIFAR-10 and CIFAR-100, some frequencies cause far more prediction changes than others, even though all perturbations satisfy the same ℓ_∞ constraint.

This structure remains stable across several independent training runs and substantially exceeds the effect of random noise of the same amplitude. However, the ϵ sweep shows that attack effectiveness depends strongly on amplitude, particularly for low-resolution images such as those in CIFAR.

Finally, the internal analysis suggests that the most sensitive frequencies are also those amplified most strongly in the network's representations. These observations do not prove that the real model behaves as a global linear operator, but they show that the frequency structure identified in the theoretical setting remains partly visible in the model's local behavior.

5 Conclusion

This work aimed to explore the intrinsic connection between the convolutional structure of neural networks (CNNs) and the emergence of preferred directions of vulnerability in the Fourier domain. More specifically, our approach sought to determine the extent to which the mathematical properties inherent in convolutional layers, beyond biases associated with the data or the training process, can explain the existence of universal adversarial perturbations.

To this end, after establishing the general framework of CNNs and adversarial attacks, we reconstructed and analyzed the theoretical reasoning proposed by Tsuzuku and Sato [14], initially using a simplified linear model.

To extend these theoretical results to actual CNN architectures, where ReLU activations and *max-pooling* operations prevent any global diagonalization, we introduced local linearization through analysis of the model's Jacobian. This theoretical extension proved crucial in justifying the application of results based on linearity to fundamentally nonlinear architectures.

Together, these observations clarify the mechanism by which a perturbation based on a single Fourier frequency can affect a convolutional network so severely. The successive amplification of this perturbation through the different layers provides a compelling structural explanation for the existence of universal perturbations.

Our experimental approach supports this intuition. The *heatmaps* obtained for `VGGSmallBN` on the CIFAR-10 and CIFAR-100 datasets reveal highly heterogeneous frequency sensitivity. Certain frequencies produce significantly higher *fool ratios*, showing that perturbations of equal amplitude do not have an equivalent impact: their direction in input space is a determining factor. Comparison with random noise of the same amplitude fully corroborates this asymmetry.

The results also highlight the expected influence of classification task complexity on model robustness. Overall, the network proves more vulnerable on CIFAR-100 than on CIFAR-10. This vulnerability, examined in greater detail in Appendix C.10, is consistent with the increased difficulty of the problem: with more classes, decision boundaries become more numerous and more closely spaced because of the visual similarity between categories, allowing a perturbation to change the prediction more easily. Nevertheless, despite this increased vulnerability, sensitivity retains a structured topology in which certain Fourier directions remain dominant.

The scope of these results should nevertheless be interpreted cautiously. Our experimental reproduction is limited to a VGG-type architecture and low-resolution images. Moreover, the most sensitive frequencies were selected from *heatmaps* computed on the test set. We therefore cannot claim that all convolutional networks exhibit exactly the same vulnerability structure. Within the setting studied here, however, Fourier directions appear to be a useful tool for better understanding certain sensitivities of CNNs.

This analysis can also be related to model explainability. When a particular frequency substantially changes the network's predictions, this indicates that this component of the image plays an important role in its decision. In other words, sensitive frequencies are useful not only for constructing adversarial perturbations: they also provide clues about the type of information the network uses to classify images.

A possible continuation of this work would therefore be to use these sensitive frequencies both to strengthen models and to interpret them more effectively. For example, they could guide new forms of data augmentation, limit certain internal amplifications, or support comparisons of more robust architectures, particularly those incorporating anti-aliasing mechanisms.

This direction would connect three important aspects: adversarial robustness, resistance to variations in the data, and interpretation of the network's decisions. The goal would therefore be not only to show that certain Fourier frequencies are dangerous for the model, but also to understand how identifying them can help build networks that are more stable, more robust, and more interpretable.

A Matrix Representation of a Linear Map

Let V and W be two finite-dimensional vector spaces over a field $\mathbb{K}$, where $\mathbb{K}$ denotes either $\mathbb{R}$ or $\mathbb{C}$. Let

$$\mathcal{B} = (e_1, \ldots, e_N)$$

be a basis of V, and

$$\mathcal{B}' = (e'_1, \ldots, e'_M)$$

a basis of W.

Now consider a linear map

$$f : V \to W.$$

Every vector $v \in V$ has a unique decomposition in the basis $\mathcal{B}$:

$$v = \sum_{i=1}^{N} \alpha_i e_i,$$

where the coefficients α_i are the coordinates of v in the basis $\mathcal{B}$.

By linearity of f,

$$f(v) = f\left(\sum_{i=1}^{N} \alpha_i e_i\right) = \sum_{i=1}^{N} \alpha_i f(e_i).$$

Since $f(e_i) \in W$, each vector $f(e_i)$ can itself be decomposed in the basis $\mathcal{B}'$. There therefore exist scalars $a_{ji} \in \mathbb{K}$ such that

$$f(e_i) = \sum_{j=1}^{M} a_{ji} e'_j.$$

Substituting this expression into the preceding formula gives

$$f(v) = \sum_{i=1}^{N} \alpha_i \left(\sum_{j=1}^{M} a_{ji} e'_j\right).$$

Rearranging the sums according to the vectors of the basis $\mathcal{B}'$ yields

$$f(v) = \sum_{j=1}^{M} \left(\sum_{i=1}^{N} a_{ji} \alpha_i\right) e'_j.$$

If we write

$$[f(v)]_{\mathcal{B}'} = \begin{pmatrix} \beta_1 \\ \vdots \\ \beta_M \end{pmatrix}, \qquad [v]_{\mathcal{B}} = \begin{pmatrix} \alpha_1 \\ \vdots \\ \alpha_N \end{pmatrix},$$

then the coordinates of $f(v)$ satisfy

$$\beta_j = \sum_{i=1}^{N} a_{ji} \alpha_i.$$

We can therefore write this in matrix form as

$$[f(v)]_{\mathcal{B}'} = A[v]_{\mathcal{B}},$$

where

$$A = (a_{ji})_{\substack{1 \le j \le M \\ 1 \le i \le N}} \in \mathbb{K}^{M \times N}.$$

In other words, the matrix A representing the linear map f is constructed column by column: its i-th column is the coordinate vector of $f(e_i)$ in the basis $\mathcal{B}'$:

$$A = \begin{pmatrix} | & | & & | \\ [f(e_1)]_{\mathcal{B}'} & [f(e_2)]_{\mathcal{B}'} & \cdots & [f(e_N)]_{\mathcal{B}'} \\ | & | & & | \end{pmatrix}.$$

Thus, every linear map between finite-dimensional vector spaces can be represented by a matrix once a basis has been chosen for the domain and a basis for the codomain.

This matrix depends on the choice of bases. The same linear map can therefore have different matrix representations depending on the bases used.

B Eigenvectors, Eigenvalues, and Diagonalization

Let V be a vector space of finite dimension N over a field $\mathbb{K}$, and let

$$f : V \to V$$

be an endomorphism, that is, a linear map from V to itself.

A nonzero vector $v \in V$ is called an eigenvector of f if there exists a scalar $\lambda \in \mathbb{K}$ such that

$$f(v) = \lambda v.$$

The scalar λ is then called the eigenvalue associated with the eigenvector v.

This relation means that the map f does not change the direction of the vector v. It simply multiplies it by a scalar factor λ.

Now consider a basis

$$\mathcal{B} = (e_1, \ldots, e_N)$$

of V. As recalled in Appendix A, the endomorphism f can be represented by a matrix in this basis.

A natural question is whether we can choose a basis in which the matrix of f becomes particularly simple.

The most favorable case is when the basis $\mathcal{B}$ consists entirely of eigenvectors of f. Suppose, therefore, that for every $i \in \{1, \ldots, N\}$,

$$f(e_i) = \lambda_i e_i.$$

Then, in the basis $\mathcal{B}$, the coordinates of $f(e_i)$ are zero everywhere except at the i-th position:

$$[f(e_i)]_{\mathcal{B}} = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ \lambda_i \\ 0 \\ \vdots \\ 0 \end{pmatrix}.$$

The matrix representing f in this basis therefore has these coordinate vectors as its columns.

It is diagonal:

$$A_{\mathcal{B}} = \begin{pmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_N \end{pmatrix}.$$

In other words, given a basis of eigenvectors, the action of f becomes very simple: each coordinate is merely multiplied by the corresponding eigenvalue.

Conversely, if the matrix of f is diagonal in a given basis, then the vectors of that basis are eigenvectors of f.

This observation explains why it is useful to find a basis of eigenvectors: in such a basis, the linear map no longer mixes directions. It acts independently on each direction of the basis.

In this report, this idea is applied to circular convolution. The matrix associated with circular convolution has the Fourier vectors as eigenvectors. Thus, when the signal is expressed in the Fourier basis, the convolution operator becomes diagonal:

$$C_h = F_N \Lambda_h F_N^{-1},$$

where F_N contains the Fourier vectors and Λ_h is the diagonal matrix of the associated eigenvalues.

C Supplementary Material

C.1 Periodicity, Boundaries, and the Gibbs Phenomenon

Throughout this thesis, we frequently use the discrete Fourier transform and circular convolution. These tools implicitly rely on a signal periodization assumption. It is therefore useful to briefly recall the relationship between periodicity, the Fourier spectrum, and boundary effects.

- If a signal is defined on an infinite domain, for example $\mathbb{R}$, and is not periodic, its frequency content is described by a *continuous spectrum*. This is the setting of the Fourier transform.
- If a signal is periodic, by contrast, its frequency content is *discrete*: the signal decomposes into a sum of harmonic modes with well-defined frequencies. This is the setting of Fourier series.
- Finally, for digital signals of finite length, a periodization assumption is often used. The signal is then considered to be periodically extended: its two endpoints are artificially joined, as if the sheet on which it is drawn were rolled into a loop.

This assumption is mathematically very useful because it allows us to use the discrete Fourier transform and to represent certain convolutions as circular convolutions. However, it can introduce an artifact when the signal boundaries do not join naturally. If the right boundary does not meet the left boundary at a compatible value, an artificial discontinuity appears at the junction.

This discontinuity matters from a Fourier perspective. An approximation using a finite number of Fourier modes struggles to represent an abrupt jump. It therefore produces spurious oscillations near the discontinuity. This behavior is known as the Gibbs phenomenon [11].

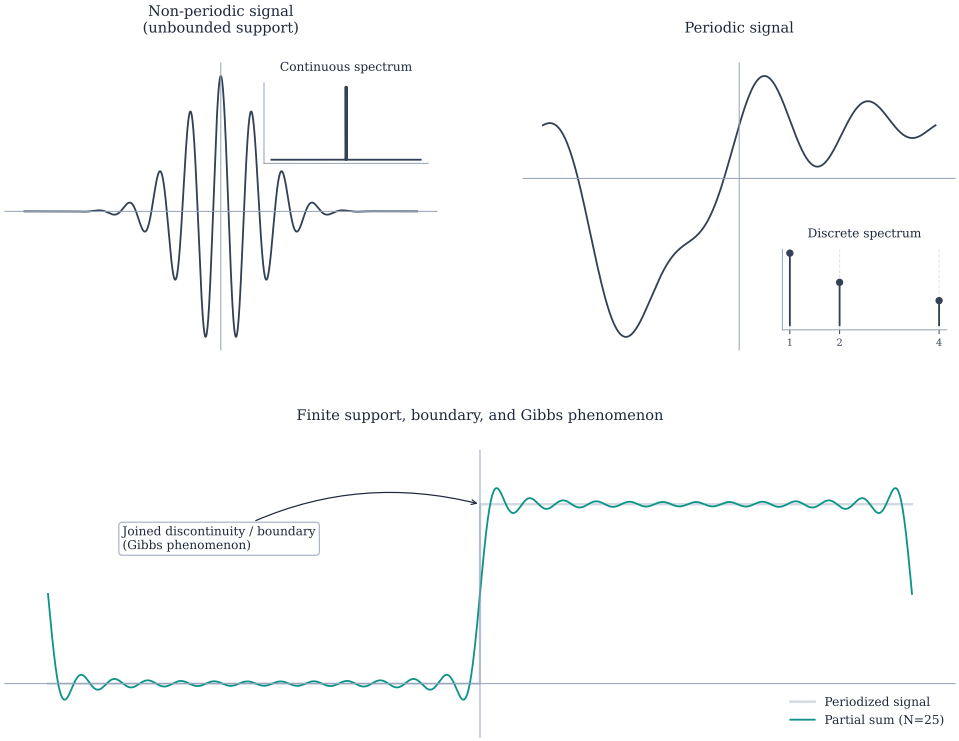


Figure C.1: Left: a nonperiodic signal defined on an infinite domain has a continuous spectrum. Center: a periodic signal has a discrete spectrum consisting of harmonic lines. Right: a signal with finite support may exhibit an artificial discontinuity when periodized.

C.2 Detailed Proof of the BCCB Structure

We revisit the case of a single-channel 2D circular convolution. The goal is to show, step by step, why the matrix M associated with this convolution is block-circulant and why each of its blocks is itself circulant.

Proof. Let $X \in \mathbb{C}^{N\times N}$ denote the input image, $Y \in \mathbb{C}^{N\times N}$ the output image, and $h \in \mathbb{C}^{N\times N}$ the filter. Recall that, in the circular setting, 2D convolution is written as

$$\begin{aligned} Y[m,n] &= \sum_{a=0}^{N-1}\sum_{b=0}^{N-1} h[a,b]\, X[(m-a) \bmod N,\, (n-b) \bmod N] \\ &= \sum_{a=0}^{N-1}\sum_{b=0}^{N-1} h[(m-a) \bmod N,\, (n-b) \bmod N]\, X[a,b]. \end{aligned}$$

With column-wise vectorization, the input entry $X[r,s]$ corresponds to the vectorized index $r+Ns$, and the output entry $Y[m,n]$ corresponds to the vectorized index $m+Nn$. The matrix M is defined by

$$\operatorname{vec}(Y) = M \operatorname{vec}(X).$$

The entry of M in row $m+Nn$ and column $r+Ns$ therefore indicates how pixel $X[r,s]$ contributes to pixel $Y[m,n]$.

According to the 2D convolution formula, this contribution is given by

$$M_{m+Nn,\, r+Ns} = h[(m-r) \bmod N,\, (n-s) \bmod N].$$

This formula is central to the proof: the entry depends only on the circular shifts between the input and output positions.

We now partition the matrix M into $N \times N$ blocks, each of size $N \times N$:

$$M = \begin{pmatrix} B_{0,0} & B_{0,1} & \cdots & B_{0,N-1} \\ B_{1,0} & B_{1,1} & \cdots & B_{1,N-1} \\ \vdots & \vdots & \ddots & \vdots \\ B_{N-1,0} & B_{N-1,1} & \cdots & B_{N-1,N-1} \end{pmatrix}.$$

In this partition, block $B_{n,s}$ describes the effect of column s of the input image on column n of the output image.

The entries of this block are therefore

$$(B_{n,s})_{m,r} = M_{m+Nn,\, r+Ns} = h[(m-r) \bmod N,\, (n-s) \bmod N].$$

Now fix two columns n and s. Their circular shift is

$$\delta = (n-s) \bmod N.$$

With this notation, the preceding expression becomes

$$(B_{n,s})_{m,r} = h[(m-r) \bmod N, \delta].$$

For a fixed δ, the entry $(B_{n,s})_{m,r}$ depends on m and r only through the circular difference $(m-r) \bmod N$. This means that the rows of the block repeat under circular shifts. In other words, each block $B_{n,s}$ is a circulant matrix.

To make this more explicit, define the vector

$$h_\delta = \begin{pmatrix} h[0,\delta] \\ h[1,\delta] \\ \vdots \\ h[N-1,\delta] \end{pmatrix}.$$

Block $B_{n,s}$ is then the circulant matrix constructed from this vector:

$$B_{n,s} = C_{h_\delta}.$$

Since $\delta = (n - s) \bmod N$, we can also write

$$B_{n,s} = C_{h_{(n-s) \bmod N}}.$$

This formula reveals the second level of circularity. As n and s change, the blocks do not vary arbitrarily: they depend only on the circular shift $(n - s) \bmod N$. The blocks themselves are therefore arranged in a circulant pattern.

The full matrix M can then be written as

$$M = \begin{pmatrix} C_{h_0} & C_{h_{N-1}} & C_{h_{N-2}} & \cdots & C_{h_1} \\ C_{h_1} & C_{h_0} & C_{h_{N-1}} & \cdots & C_{h_2} \\ C_{h_2} & C_{h_1} & C_{h_0} & \cdots & C_{h_3} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ C_{h_{N-1}} & C_{h_{N-2}} & C_{h_{N-3}} & \cdots & C_{h_0} \end{pmatrix}.$$

This representation clearly exhibits the two required properties:

each block is circulant and the blocks themselves form a circulant pattern.

Consequently, M is indeed a block-circulant matrix with circulant blocks, that is, a BCCB matrix.

□

C.3 Detailed Proof for Skip Connections

We revisit the identity *skip connection* studied in Section 3.9. The goal is to verify explicitly that the right singular vectors retain the same frequency structure as in the case without a *skip connection*.

Proof. Let $a \in \mathbb{C}^m$ be a right singular vector of $G^{(w)}_{\text{skip}}$, associated with a singular value σ and a left singular vector $b \in \mathbb{C}^m$:

$$G^{(w)}_{\text{skip}} a = \sigma b.$$

In the full network space, consider the vector

$$v = a \otimes (Q_N)_w.$$

In the same notation as before, this corresponds to

$$v = a \otimes (F_N)_u \otimes (F_N)_v,$$

where the flattened index w corresponds to the spatial frequency pair (u, v).

Now apply the operator M_{skip} to this vector:

$$M_{\text{skip}} v = (I_m \otimes Q_N)(L + I_{mN^2})(I_m \otimes Q_N)^H \big(a \otimes (Q_N)_w\big).$$

Since $(Q_N)_w = Q_N e_w$, we have

$$a \otimes (Q_N)_w = (I_m \otimes Q_N)(a \otimes e_w).$$

Thus,

$$M_{\text{skip}} v = (I_m \otimes Q_N)(L + I_{mN^2})(a \otimes e_w).$$

The standard basis vector e_w selects the coefficients associated with frequency w. At this frequency, the central operator acts as the augmented mixing matrix

$$G_{\text{skip}}^{(w)} = G^{(w)} + I_m.$$

Thus, its action on the channels separates from its action on the spatial frequency:

$$\begin{aligned}(L + I_{mN^2})(a \otimes e_w) &= (G^{(w)} a + I_m a) \otimes e_w \\ &= (G_{\text{skip}}^{(w)} a) \otimes e_w.\end{aligned}$$

Since a is a right singular vector of $G_{\text{skip}}^{(w)}$, we have

$$G_{\text{skip}}^{(w)} a = \sigma b.$$

We therefore obtain

$$(L + I_{mN^2})(a \otimes e_w) = \sigma(b \otimes e_w).$$

Substituting this result into the expression for $M_{\text{skip}} v$ gives:

$$\begin{aligned}M_{\text{skip}} v &= (I_m \otimes Q_N)\big(\sigma(b \otimes e_w)\big) \\ &= \sigma\big((I_m b) \otimes (Q_N e_w)\big) \\ &= \sigma\big(b \otimes (Q_N)_w\big).\end{aligned}$$

Thus, applying M_{skip} to the vector

$$v = a \otimes (Q_N)_w$$

indeed yields an output vector with the same spatial frequency:

$$M_{\text{skip}} v = \sigma\big(b \otimes (Q_N)_w\big).$$

The spatial frequency $(Q_N)_w$ is therefore preserved. Only the overall amplitude, given by σ, and the channel profile, given by b, change.

Consequently, the right singular vectors of the operator with a *skip connection* remain of the form

$$a \otimes (Q_N)_w,$$

that is,

$$a \otimes (F_N)_u \otimes (F_N)_v.$$

□

C.4 Graphical Representation of Stride

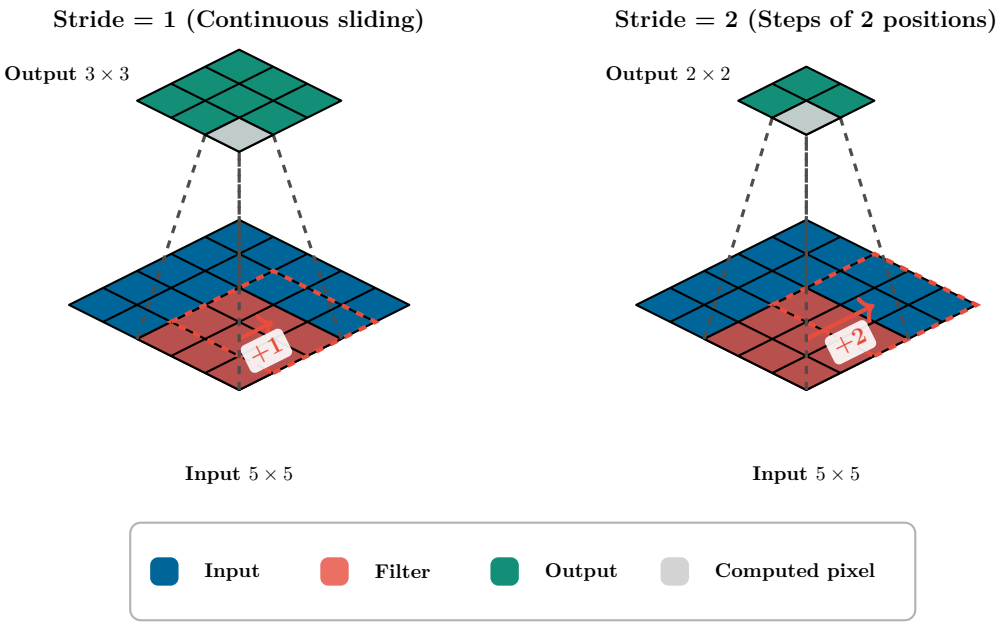


Figure C.2: Effect of *stride* on the output spatial resolution.

C.5 Illustration of Average Pooling

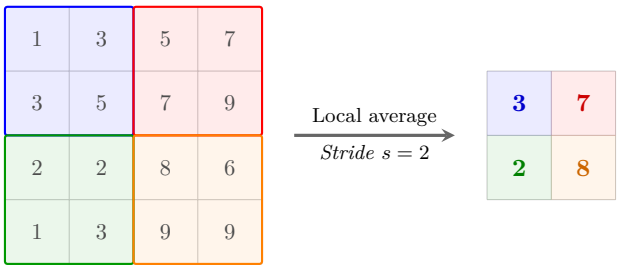


Figure C.3: Spatial illustration of the *average pooling* operation. Each 2×2 block of pixels in the input image is replaced by its arithmetic mean, reducing the spatial resolution by a factor of $s = 2$.

C.6 Review of the SVD and Amplification Directions

Let

$$M \in \mathbb{C}^{m_{\text{out}}N^2 \times m_{\text{in}}N^2}$$

be the matrix representing a multichannel convolution operator. Since this matrix is not necessarily square, we use singular value decomposition rather than standard eigendecomposition.

The singular value decomposition of M is

$$M = U\Sigma V^H,$$

where U and V are unitary matrices, and Σ is a rectangular diagonal matrix containing the singular values of M:

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq 0.$$

The columns of V are called the right singular vectors of M: they lie in the input space. The columns of U are called the left singular vectors of M: they lie in the output space.

To understand why right singular vectors describe amplification directions, consider the positive self-adjoint matrix $M^H M$. From the SVD, we obtain

$$M^H M = (U\Sigma V^H)^H (U\Sigma V^H).$$

Using $(ABC)^H = C^H B^H A^H$ gives

$$M^H M = V\Sigma^H U^H U\Sigma V^H.$$

Since U is unitary, $U^H U = I$, so

$$M^H M = V\Sigma^H \Sigma V^H.$$

The matrix $\Sigma^H\Sigma$ is diagonal and contains the squared singular values:

$$\Sigma^H\Sigma = \operatorname{diag}(\sigma_1^2, \sigma_2^2, \dots).$$

Thus,

$$M^H M = V \operatorname{diag}(\sigma_1^2, \sigma_2^2, \dots) V^H.$$

This shows that the right singular vectors of M are precisely the eigenvectors of $M^H M$, and that the associated eigenvalues are the squared singular values.

In particular, if v_i denotes the i-th column of V, then

$$M^H M v_i = VDV^H v_i = VDe_i = V(\sigma_i^2 e_i) = \sigma_i^2 v_i,$$

where

$$D = \operatorname{diag}(\sigma_1^2, \sigma_2^2, \dots).$$

Thus,

$$M^H M v_i = \sigma_i^2 v_i.$$

Equivalently, the singular value σ_i measures the amplification induced by M in direction v_i. Indeed,

$$\|Mv_i\|_2^2 = (Mv_i)^H(Mv_i) = v_i^H M^H M v_i = v_i^H(\sigma_i^2 v_i) = \sigma_i^2 \|v_i\|_2^2.$$

Since V is unitary, its columns form an orthonormal basis. In particular,

$$\|v_i\|_2 = 1.$$

We therefore obtain

$$\|Mv_i\|_2 = \sigma_i.$$

The largest singular value describes the maximum amplification of M among inputs with norm 1:

$$\sigma_1 = \max_{\|x\|_2=1} \|Mx\|_2.$$

If σ_1 is simple, the direction achieving this maximum amplification is the right singular vector v_1. If σ_1 has multiplicity greater than one, there is no single maximizing direction: every unit-norm direction in the subspace associated with σ_1 yields the same amplification.

In the frequency analysis of convolutions, the network's highly sensitive directions therefore correspond to right singular vectors associated with large singular values.

C.7 Mean Fourier Spectrum of Clean Images

This appendix presents the mean Fourier spectrum of unperturbed CIFAR-10 and CIFAR-100 images. The aim is not to measure model sensitivity directly, but to observe which frequencies are naturally present in the input data.

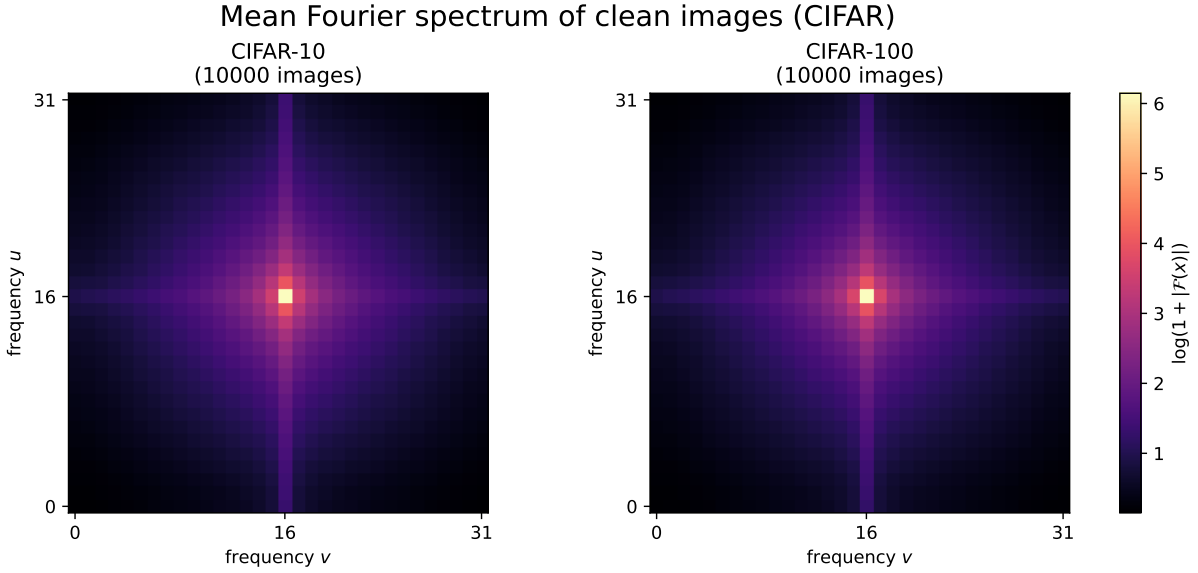


Figure C.4: Mean Fourier spectrum of unperturbed CIFAR-10 and CIFAR-100 images. The displayed values correspond to $\log(1 + |\mathcal{F}(x)|)$, averaged over the images in the dataset.

The two datasets exhibit very similar frequency structures: energy is strongly concentrated

around the center of the spectrum and gradually decreases in an approximately diamond-shaped pattern. The more pronounced horizontal and vertical lines also indicate that certain frequency directions are more prevalent than others in the unperturbed images.

This observation suggests a possible interpretation of the SFA heatmaps. It shows that the input images themselves have a nonuniform frequency organization. Model sensitivity may therefore depend both on the architecture and on the statistics of the data used during training.

However, this comparison remains descriptive. The mean spectrum characterizes the input images, whereas the SFA heatmaps measure the model's response to imposed perturbations. The two should therefore not be conflated.

C.8 Supplementary Analysis: The Role of Channel Mixing in SFA

In the theoretical framework presented in the previous chapter, the singular directions of a linearized convolutional model can be written in the form

$$v = a \otimes q_{u,v},$$

where $q_{u,v}$ represents a spatial Fourier component and $a \in \mathbb{R}^3$ describes the mixing of the red, green, and blue channels.

The *Single Fourier Attack* used in the main protocol simplifies this framework. It does not seek to optimize the spatial frequency and the color vector a simultaneously. The same Fourier pattern is applied to all three RGB channels, which amounts to using the vector

$$a_{\text{uni}} = (1, 1, 1).$$

The search space is therefore reduced to the choice of frequency (u, v). This simplification makes the attack much easier to evaluate, since we need only iterate over the frequencies in the Fourier domain. It is also consistent with the ℓ_∞ constraint used in the main protocol: the pattern is applied at the maximum allowed amplitude to each of the three channels.

We then distinguish two ways of constraining channel mixing. The first retains an ℓ_∞-norm constraint comparable to standard SFA, to test whether another choice of color direction can improve the attack at the same maximum amplitude. The second uses an ℓ_1-type budget on the color vector's coefficients. In this case, a vector such as $a = (3, 0, 0)$ is possible: the total budget is concentrated in a single channel, which no longer corresponds exactly to the standard SFA constraint. The two variants therefore do not measure exactly the same thing; rather, they test whether the choice of RGB mixing can strengthen the effect of a Fourier frequency.

Searching for RGB Vectors at Each Frequency

Figure C.5 compares standard SFA with variants in which the RGB vector a is chosen from a limited set of possible combinations.

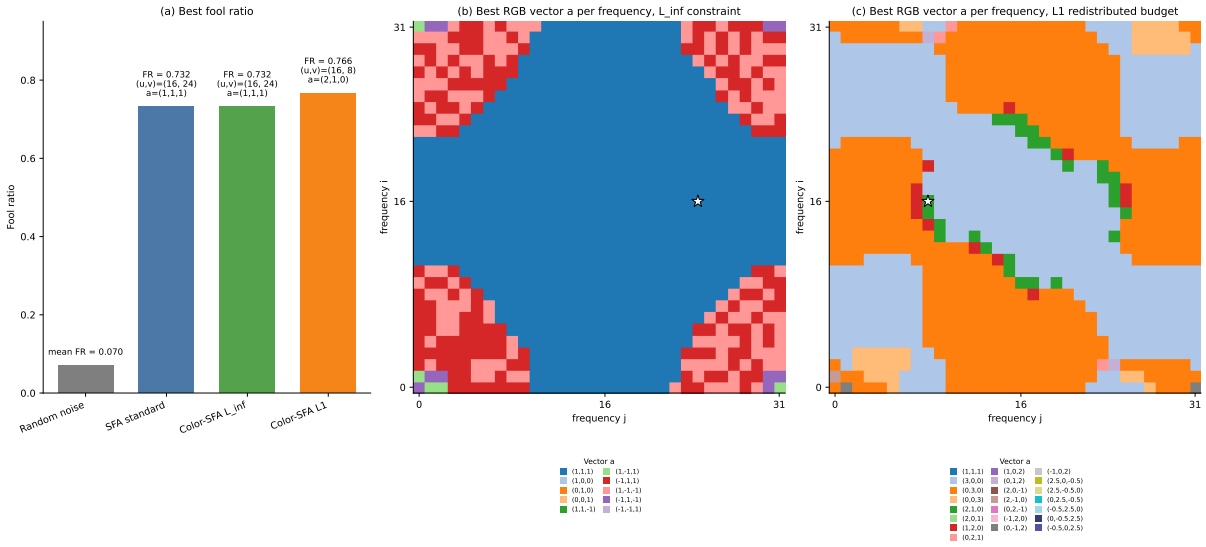


Figure C.5: Comparison between standard SFA and variants in which the RGB vector a is chosen for each frequency. Left: the best *fool ratio* obtained by each method is compared with the random-noise baseline. Center and right: each cell indicates the best RGB vector found for the corresponding frequency under two different budget constraints.

In the variant renormalized in the ℓ_∞ norm, the best color variant achieves almost the same result as standard SFA. This indicates that, under this constraint, choosing a different RGB mixture does not significantly improve the attack compared with the uniform direction $(1, 1, 1)$.

With the variant using an ℓ_1 budget on the color coefficients, the best score increases slightly, from approximately 0.732 to 0.766, with frequency $(16, 8)$ and vector $a = (2, 1, 0)$. This improvement is fairly modest in our case.

Comparison of Fixed Color Directions

We then compared several simple color directions:

$$(1, 1, 1), \quad (1, 0, 0), \quad (0, 1, 0), \quad (0, 0, 1),$$

as well as opponent color directions:

$$(1, -1, 0), \quad (1, 0, -1), \quad (0, 1, -1),$$

and a luminance-like direction.

The results in Figure C.6 show that the standard direction $(1, 1, 1)$ remains by far the most effective. Directions applied to a single channel have a weaker effect, while the opponent color directions $(1, -1, 0)$, $(1, 0, -1)$, and $(0, 1, -1)$ yield almost zero fool ratios.

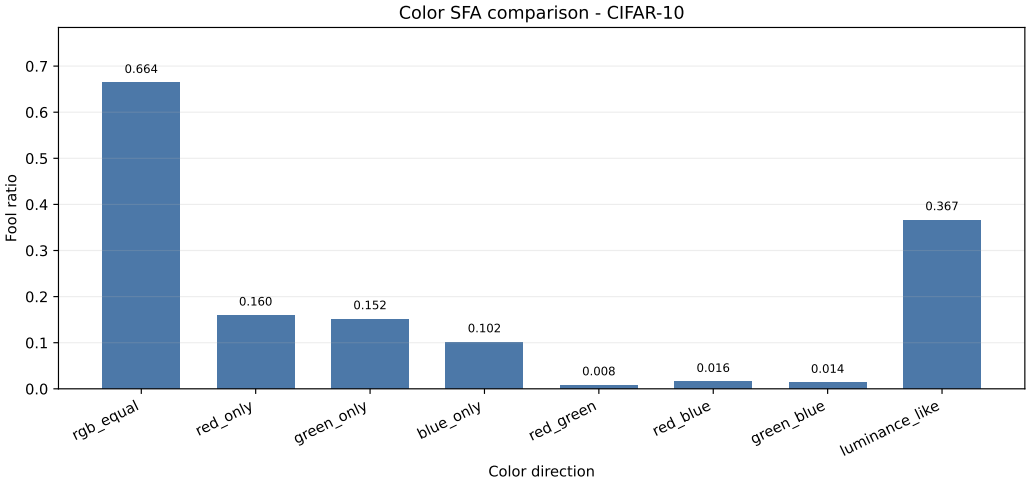


Figure C.6: Comparison of several color directions for the *Single Fourier Attack* on CIFAR-10.

This observation suggests that, for the frequency studied, opponent color directions are weakly transmitted by the network. We can interpret this using the multichannel framework. For a fixed frequency (u, v), the effect of a color direction $a \in \mathbb{R}^3$ is described by the mixing matrix $G^{(u,v)}$:

$$\beta = G^{(u,v)} a.$$

Standard SFA corresponds to the choice

$$a_{\text{uni}} = (1, 1, 1),$$

that is, a common variation in all three RGB channels.

Ideally symmetric case. Suppose that, at the frequency considered, the network responds identically to the three input channels. This amounts to assuming that the three columns of $G^{(u,v)}$ are identical:

$$G^{(u,v)} = \begin{pmatrix} | & | & | \\ g & g & g \\ | & | & | \end{pmatrix} = g \begin{pmatrix} 1 & 1 & 1 \end{pmatrix},$$

where $g \in \mathbb{C}^{m_{\text{out}}}$ describes the network's common frequency response.

Then, for any color direction $a = (a_R, a_G, a_B)$, we obtain

$$G^{(u,v)} a = g(a_R + a_G + a_B).$$

Amplification therefore depends only on the sum of the components of a. Under an ℓ_∞ constraint comparable to that of standard SFA,

$$|a_R|, |a_G|, |a_B| \leq 1,$$

the absolute value of this sum is maximized by

$$a = (1, 1, 1)$$

or its negative. Thus, in this simplified symmetric setting, the uniform direction a_{uni} is optimal.

Conversely, any color direction satisfying

$$a_R + a_G + a_B = 0$$

is annihilated by $G^{(u,v)}$. This is the case for the opponent color directions

$$(1, -1, 0), \qquad (1, 0, -1), \qquad (0, 1, -1).$$

In this simplified model, these directions therefore belong to the kernel of the per-frequency mixing matrix:

$$G^{(u,v)}a = 0.$$

Approximately symmetric case. In a real network, the columns of $G^{(u,v)}$ are not exactly identical. However, we can separate a component common to all three channels from a residual term:

$$G^{(u,v)} = g \begin{pmatrix} 1 & 1 & 1 \end{pmatrix} + E,$$

where we can take

$$g = \frac{1}{3} G^{(u,v)} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}.$$

The term E then measures the deviation from a response that is perfectly shared by all three channels.

For an opponent color direction such that $a_R + a_G + a_B = 0$, the common component vanishes:

$$G^{(u,v)}a = Ea.$$

Thus, if $\|E\|$ is small, then

$$\|G^{(u,v)}a\|_2 \leq \|E\| \, \|a\|_2.$$

Opponent color directions are therefore no longer exactly annihilated, but they remain weakly transmitted when frequency-dependent channel mixing is close to a common response.

This proposition provides a theoretical interpretation of the results observed in Figure C.6. The high effectiveness of $(1, 1, 1)$ suggests that the frequency studied primarily activates a direction common to the channels, while the low scores of opponent color directions may be explained by near cancellation in frequency-dependent channel mixing.

C.9 Internal Heatmaps by Layer

To complement the amplification analysis presented in the main chapter, we visualize here the effect of Fourier perturbations at different internal levels of the network. For each frequency, we measure the normalized mean change in activations after perturbation and then compare it with the final *fool ratio*.

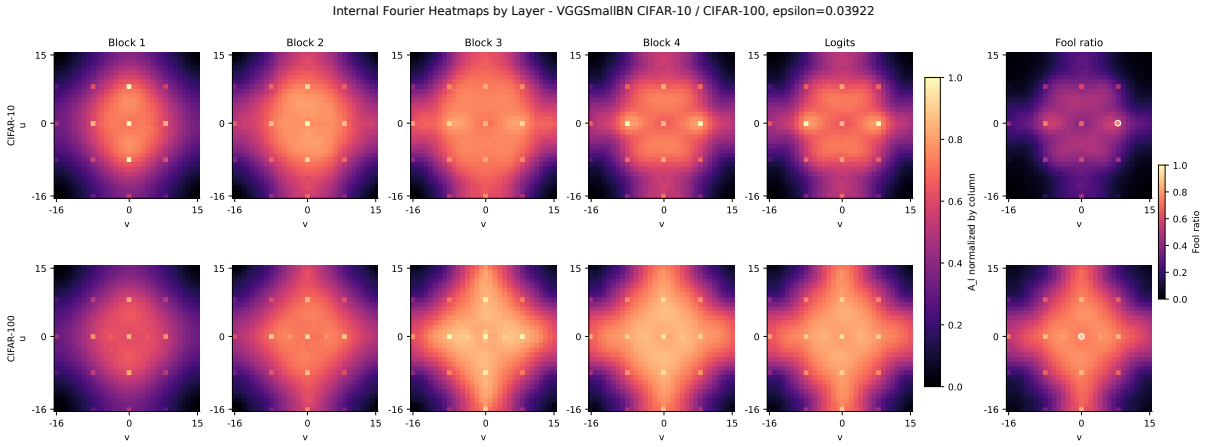


Figure C.7: Internal heatmaps by layer for `VGGSmallBN` on CIFAR-10 and CIFAR-100. The first columns show the normalized mean change in internal activations after Fourier perturbation. The last column shows the final *fool ratio*.

These visualizations show that the frequency structure observed in the final heatmaps is already present in the network's internal representations. It becomes more pronounced in deeper blocks and in the logits, suggesting that certain frequencies are progressively amplified during convolutional processing.

C.10 Further Analysis of Logit Margins and Local Boundaries

This appendix complements the experimental analysis in Chapter 4. The goal is to better understand why the model trained on CIFAR-100 appears more fragile than the one trained on CIFAR-10.

C.10.1 Distribution of Logit Margins

For each image in the test set, we compute the logits produced by the model. Let $z_{(1)}$ denote the largest logit and $z_{(2)}$ the second-largest logit. The logit margin is defined as

$$m(x) = z_{(1)}(x) - z_{(2)}(x).$$

This quantity simply measures the gap between the predicted class and its closest competing class according to the model.

Figure C.8 compares the distributions of logit margins for VGGSmallBN on CIFAR-10 and CIFAR-100. The CIFAR-100 distribution is much more concentrated at small margins. The

median is approximately 4.15 for CIFAR-100, compared with approximately 11.12 for CIFAR-10. This means that, on CIFAR-100, the model's predictions are, on average, less strongly separated from their second-best class.

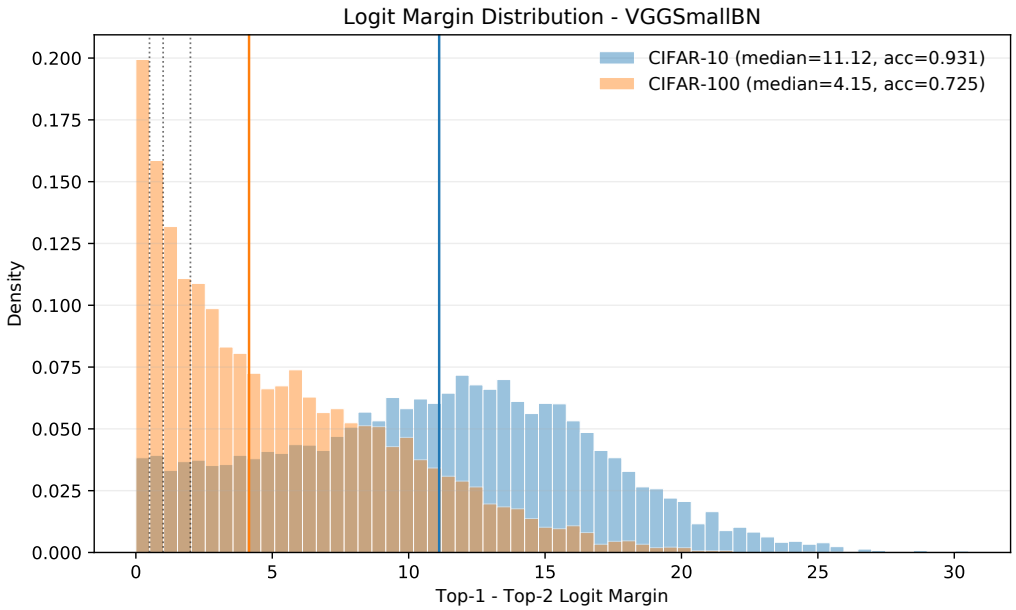


Figure C.8: Distribution of logit margins for VGGSmallBN on CIFAR-10 and CIFAR-100. The margin is defined as the difference between the largest and second-largest logits.

This result provides further support for the interpretation proposed in Chapter 4: the greater fragility observed on CIFAR-100 arises not only from the choice of a particular frequency, but also from the apparently weaker separation between the model's decisions.

Bibliography

[1] Randall Balestriero and Richard G. Baraniuk. A spline theory of deep learning. In *Proceedings of the 35th International Conference on Machine Learning*, pages 374–383. PMLR, 2018.

[2] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing.* Prentice Hall, 3 edition, 2008.

[3] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations*, 2015.

[4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

[5] Roger A. Horn and Charles R. Johnson. *Matrix Analysis.* Cambridge University Press, 2 edition, 2013.

[6] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 448–456. PMLR, 2015.

[7] Anil K. Jain. *Fundamentals of Digital Image Processing.* Prentice-Hall, 1989.

[8] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[9] Guido F. Montúfar, Razvan Pascanu, Kyunghyun Cho, and Yoshua Bengio. On the number of linear regions of deep neural networks. In *Advances in Neural Information Processing Systems*, volume 27, 2014.

[10] Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, Omar Fawzi, and Pascal Frossard. Universal adversarial perturbations. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 1765–1773, 2017.

[11] Alan V. Oppenheim and Ronald W. Schafer. *Discrete-Time Signal Processing.* Pearson, 3 edition, 2009.

[12] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations*, 2015.

[13] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. In *International Conference on Learning Representations*, 2014.

[14] Yusuke Tsuzuku and Issei Sato. On the structural sensitivity of deep convolutional networks to the directions of fourier basis functions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019.

[15] S. Voloshynovskiy. Theme 7 block 2. In *Universite de Geneve*, page 52.